CARNET

# System Architecture Document

How the system is put together, how it processes data, and what the database looks like — with the interface photographed from the running application.

Built on 14 September 2026, after Phase 32.

128 tables, 63 enumerations, 1967 columns; 37 migrations.

*Regenerated with every change: npm run docs:all. Diagrams are read from the Prisma schema, screenshots from the running app, counts from the code. Nothing in this document is typed twice.*

# Contents

1. Purpose, and how to read this document
2. The system in one picture
3. Runtime architecture
4. How a request is processed
5. How data is written
6. The workflows, as data flows
7. Database architecture, domain by domain
8. Security in the data path
9. The user interface: wireframes and photographs
10. What verifies it
Appendix A. Every table, by domain
Appendix B. Migrations
Appendix C. Rebuilding this document

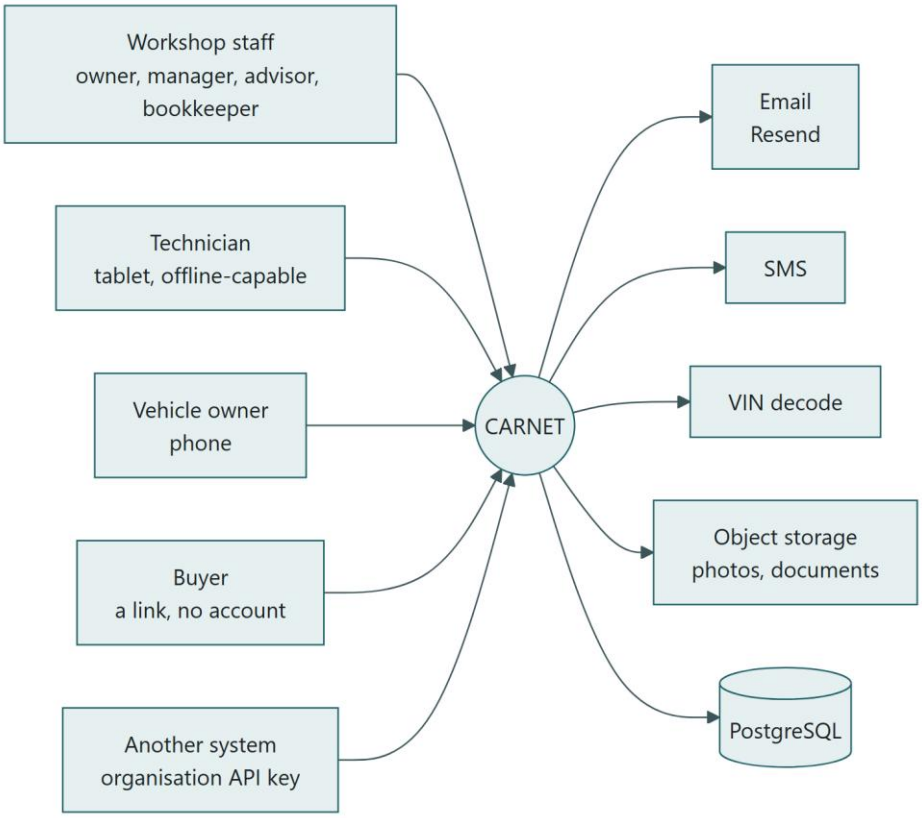
# 1. Purpose, and how to read this document

This is the one document that explains how Carnet works as a system: what runs where, what happens to a request from the moment a browser sends it to the moment a row is written, how the twelve domains of the database fit together, and what the person at the screen sees while all of that happens. It is written for an engineer joining the project, a reviewer deciding whether to trust it, and a buyer who wants to know what they are buying.

It is generated, not maintained by hand. The entity-relationship diagrams are drawn from packages/db/prisma/schema.prisma by infra/scripts/architecture-diagrams.ts; a model added to the schema appears in the next build, and one missing from the domain table fails the build rather than going quietly missing. The screenshots are taken from the running application by infra/scripts/design-shots.ts. The prose lives in infra/scripts/build_architecture_doc.py beside the code it describes and is revised with each phase.

Where this document says how something works, the source of truth is named so it can be checked: a file, a function, or a design document in docs/02-design. Where those documents go deeper — the security model, the accounting model, the state machines — this one summarises and points.

# 2. The system in one picture



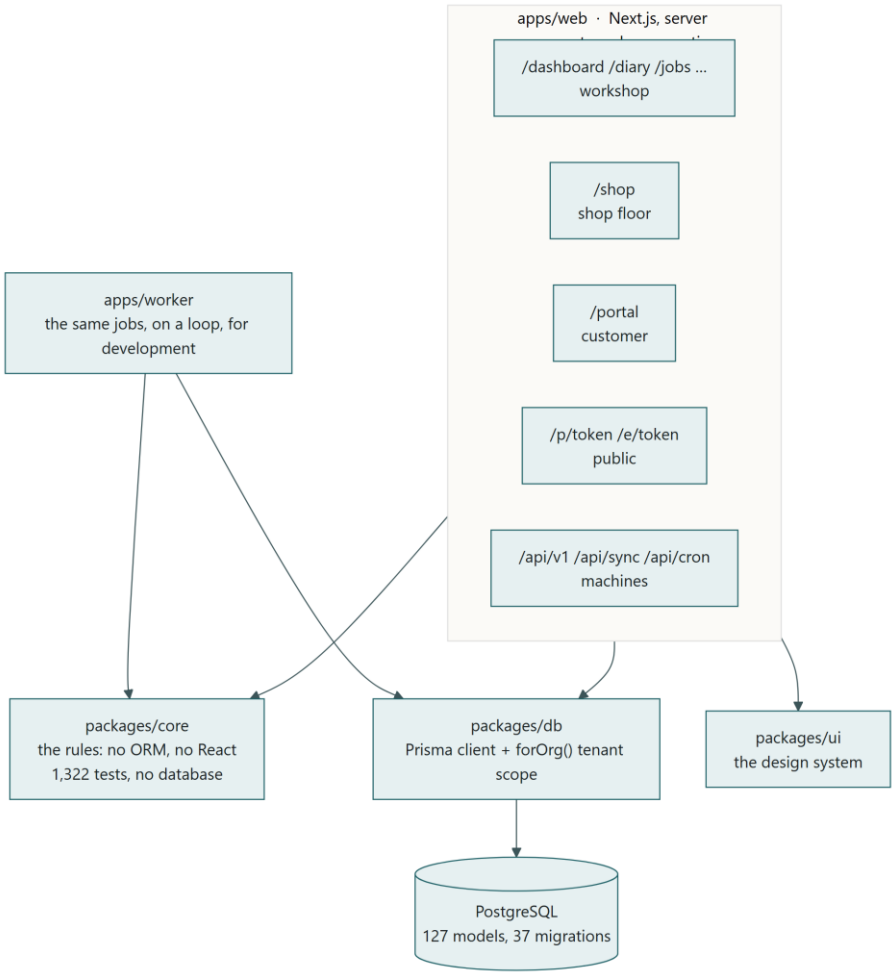

*Who uses Carnet, and what it depends on.*

Five kinds of people reach the system, and each has a surface of their own rather than the same application with items greyed out (ADR-0009). Workshop staff use the workshop at /dashboard and its siblings. A technician uses /shop on a tablet, and it keeps working out of signal. A vehicle owner uses

/portal on a phone. A buyer opens a passport link at /p/<token> with no account at all. Another system reads /api/v1 with an organisation key.

Every external dependency is optional to the core workflow. If email, SMS or VIN decoding is down, a workshop can still book, inspect, estimate, repair, invoice and take payment. The only thing the system cannot run without is PostgreSQL.

# 3. Runtime architecture




*The containers: one web application, three packages, a worker for development, one database.*

Carnet is a monorepo with one deployable web application and three packages it is built from (ADR-0003). The rules live in packages/core, which imports nothing from the rest of the repository: no ORM, no React, no framework. That is what makes the domain testable without a database — 1,322 tests run in under a second — and portable if the framework changes. packages/db is the Prisma client with the tenant scope built in; packages/ui is the design system, plain React with no data access.

## Layering

| Layer | May import | Must not import |
|---|---|---|
| `packages/core` | nothing from this repository | Prisma, React, Next |
| `packages/db` | core | React, Next |

| | | |
|---|---|---|
| `packages/ui` | core, React | Prisma, Next |
| `apps/web` | every package | — |
| `apps/worker` | core, db | React, ui |

## Technology

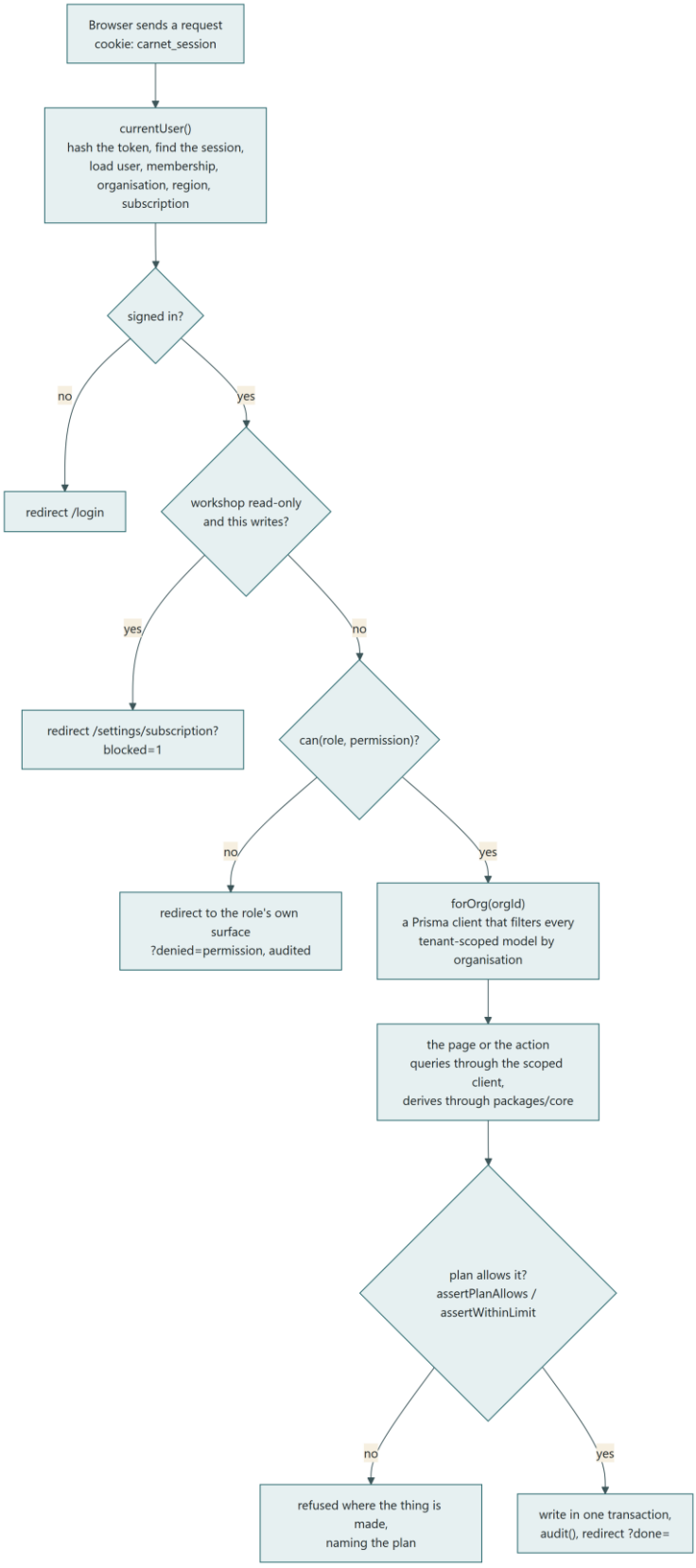
| Concern | Choice | Why |
|---|---|---|
| `Web framework` | Next.js, app router, server components | The workshop UI ships almost no JavaScript, which is the difference between a list and a spinner on a tablet over patchy Wi-Fi. |
| `Writes` | Server actions, one per form | The form posts to the function that owns the rule; no API layer to keep in step with the page. |
| `Database` | PostgreSQL through Prisma | Row locks, transactions, check constraints and JSON where the shape varies. |
| `Rules` | TypeScript in packages/core, tested with Vitest | Pure functions over plain values; no test touches the database. |
| `Sessions` | First-party, a row per session | Revocable on the server; a stolen database yields no usable cookie (ADR-0006). |
| `Email` | Resend, with inbound webhooks | Delivery events and replies come back to the job they are about. |
| `Files` | Object storage behind signed URLs | Photographs and documents never pass through the application server twice. |
| `Scheduled work` | HTTP endpoints called by a scheduler | The application is hosted where a loop cannot be held open; the worker runs the same functions in development. |
| `Verification` | tsx scripts against a built server and a real database | Fifty scripts, one per capability, each cleaning up after itself. |

## Repository layout

```
apps/web/src/app/(workshop)/*      the workshop pages and their server actions
apps/web/src/app/shop, portal, p   the shop floor, the customer's side, the public passport
apps/web/src/app/api/*             sync, cron, v1, uploads, inbound webhooks, reports
apps/web/src/lib/*                 one module per capability: money.ts, jobs.ts, passport.ts,
billing.ts …
apps/web/src/lib/guard.ts          requireSession(permission): the one door every page goes
through
packages/core/src/*                the rules, by domain: accounting, billing, passport, region,
giftcards …
packages/db/prisma/schema.prisma   the database, and migrations/ beside it, hand-written SQL
packages/db/src/tenant.ts          forOrg(): the tenant scope
packages/ui/src/*                  the design system
infra/scripts/verify-*.ts          the verification scripts
docs/                              product, requirements, design, ADRs, engineering, project
```

# 4. How a request is processed

Every page and every server action in the workshop goes through one function, requireSession, and that function is where the system decides who is asking, what they may do, where their workshop stands, and which rows they may see. The diagram is the whole path; the text after it is each step in detail.



*A request, from cookie to redirect. apps/web/src/lib/guard.ts and auth.ts.*

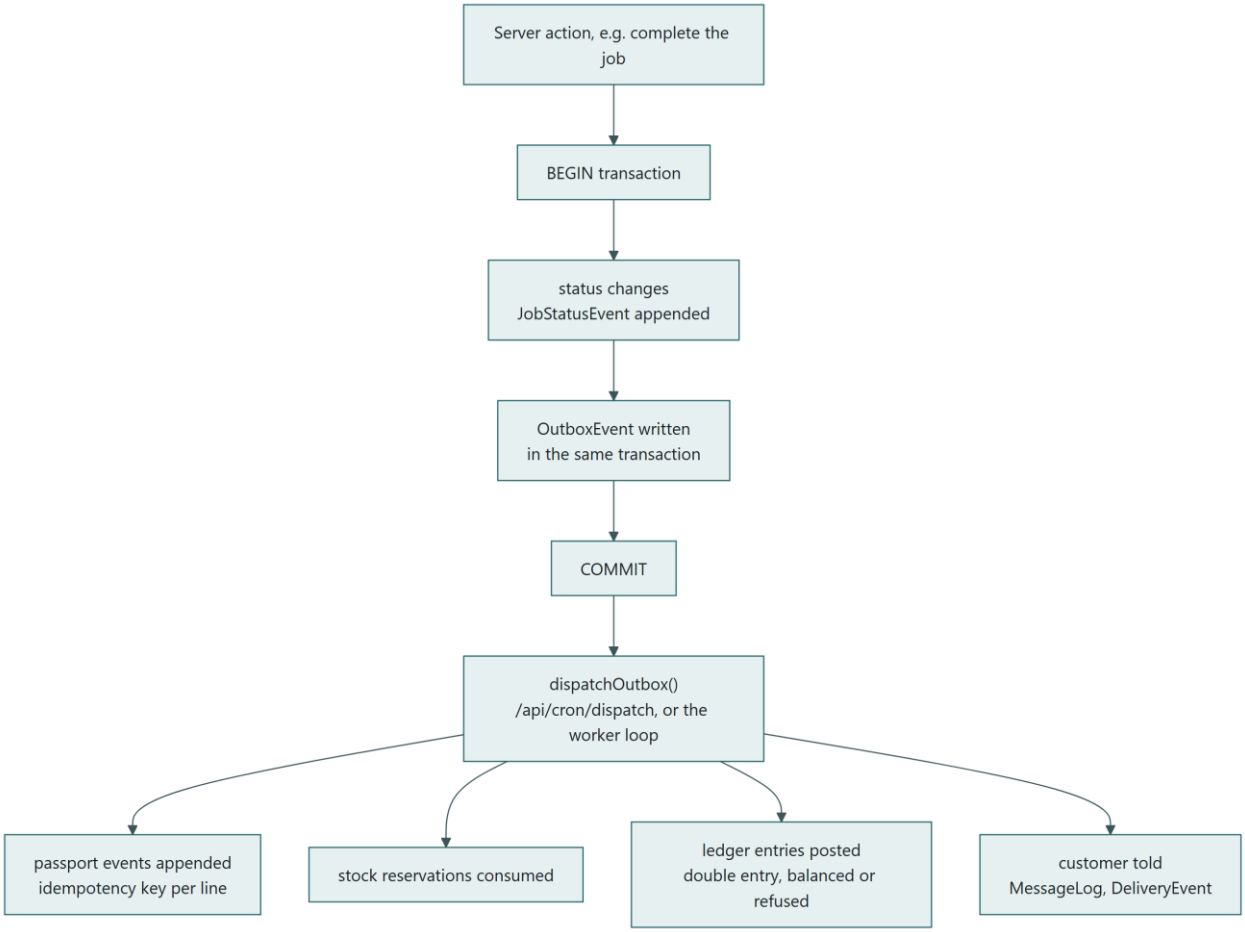
## Step by step

- The cookie. The browser sends carnet_session, a random 256-bit token. It is never stored: its SHA-256 digest is the key of the Session row. currentUser() hashes it, finds the session, checks it has not expired, and loads the user with their membership, the organisation, the organisation's region and its four subscription dates. This runs once per request and is cached for the rest of it with React's cache().
- The subscription. The organisation's state is read from its dates — trial, grace, on a plan, read-only, cancelled — and carried on the session. If the workshop is read-only and the permission asked for is a writing one (anything ending in write, complete, send, approve, request, transfer, share or manage), the request is sent to /settings/subscription with the reason before anything else is looked at.
- The permission. can(role, permission) is deny-by-default: a permission not explicitly granted to a role is refused. A refusal does not throw, because by then the page has started streaming and a thrown error would arrive as a 200 with a stack trace behind it; it redirects to the role's own surface with ?denied= and the permission, and the refusal is audited. A technician never reaches a workshop page at all: the workshop layout turns them back to /shop first.
- The scope. forOrg(orgId) returns a Prisma client where every tenant-scoped model has the organisation predicate added to reads and stamped on creates; findUnique is rewritten to findFirst with the predicate, so an id lookup cannot escape the tenant. The page receives this client as db and there is no convenient way to get an unscoped one, so the safe path is also the shortest. A row belonging to another organisation is not found, never refused, so its existence is not disclosed.
- The page. A server component queries through db, derives anything computed through packages/core (totals, statuses, balances are never stored, ADR-0011), formats money and dates from the organisation's region, and renders on the server. The browser receives HTML.
- The action. A form posts to a server action in the same file. The action calls requireSession with the permission the write needs, calls one function in apps/web/src/lib that owns the rule, and redirects back to the page with ?done=, ?saved= or ?problem= so the outcome is a sentence on the screen and the URL is bookmarkable. A rule that refuses throws a typed error with a sentence in it; the action catches that type and shows the sentence, and lets anything else surface.
- The plan. Inside the library function, not on the page, assertPlanAllows and assertWithinLimit refuse a feature the plan lacks or one more of a thing the plan counts, naming the plan that would allow it. The page's own wall is a courtesy; this is the lock.

## The other surfaces

The customer portal goes through requireUser rather than requireSession: a portal user has no organisation, and what they may see is decided by ownership (VehicleOwnership) rather than membership. The public passport at /p/<token> has no session at all; the token is the PassportShare row and its scope and expiry decide what is rendered, and every opening is recorded. /api/v1 authenticates an organisation key (hashed at rest, shown once) and reuses the same library functions the pages use, so the API and the page cannot disagree about what a workshop may see. /api/cron requires a bearer secret compared in constant time, and refuses if no secret is configured, so a misconfigured deployment fails closed.

# 5. How data is written




*A consequential write: strongly consistent inside the transaction, eventually consistent after it.*

Completing a job is the most consequential write in the system, and its shape is the shape of every write that has side effects. Inside one transaction the status changes, the JobStatusEvent is appended, and an OutboxEvent is written saying what happened. The commit is the only thing that must succeed, because a workshop losing a completed job is unacceptable. Everything downstream — passport events, stock reservations consumed, ledger entries posted, the customer told — is done by dispatchOutbox, at-least-once, with an idempotency key on every effect so a retry cannot post twice (ADR-0007).

## The conventions every write follows

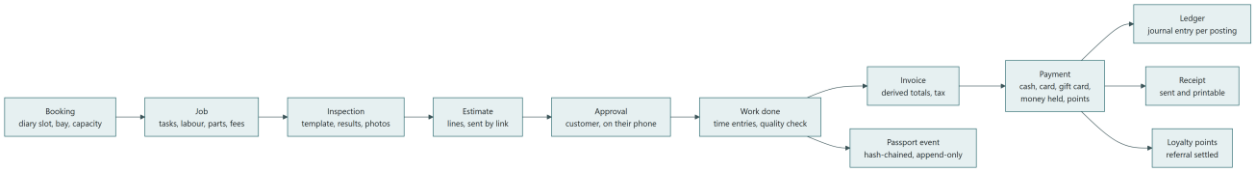
- Money is Decimal(12,2) in the database and integer minor units in the rules. fromDecimal converts at the boundary, with the currency, and sum() refuses to combine two currencies. No arithmetic on money happens in floating point (ADR-0008).
- Totals, balances and statuses are derived, never stored, wherever a stored figure could drift from the rows beneath it: an invoice's total from its lines, a gift card's balance from its entries, money held from its movements, a subscription's state from its dates (ADR-0011).
- Every ledger posting is a balanced JournalEntry with lines, refused before it is written if the debits and credits differ, and refused if the period is closed. Reversals are new entries that point at what they reverse. The accounting model in docs/02-design/07-accounting-model.md lists every posting.
- Append-only tables are never updated or deleted: VehicleEvent, JobStatusEvent, AuditLog, GiftCardEntry, DepositMovement, DeliveryEvent, ConsentChange. A correction is a new row that supersedes an old one.

- Gapless numbers (invoices, credit notes, purchase orders) come from DocumentSequence under a row lock, one sequence per organisation and kind.
- Concurrency on the passport is a row lock on the vehicle, because two workshops completing jobs on the same car in the same second must not both claim the same sequence number.
- Every consequential action writes an AuditLog row with who, what, before and after. The audit write never throws: a failure to audit must not roll back the action it describes.
- Every message to a customer is a MessageLog row first, so what was said and when is a query, and the send itself is a side effect that can fail and be retried without saying it twice.

# 6. The workflows, as data flows

This section follows the data through the workshop's work: what is created, what it points at, and what it causes. Each subsection names the tables and the module that owns the rule.

## 6.1 From a booking to a paid bill

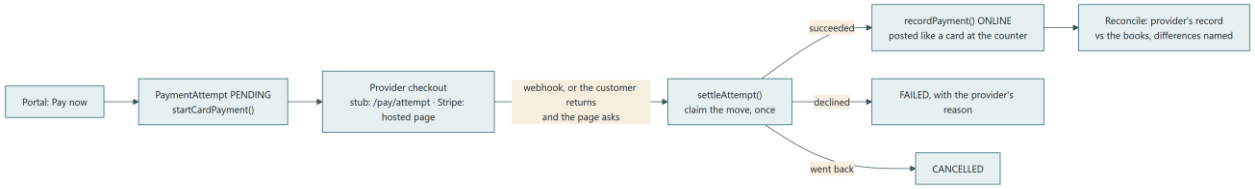



*The core workflow. Booking → Job → Inspection → Estimate → Approval → Invoice → Payment → Ledger and Passport.*

A Booking takes a diary slot against a bay after the capacity check in packages/core/src/scheduling agrees there is room. Converting it makes a Job with the vehicle, the customer and the job types, and a StockReservation for any part already on it. An Inspection runs the workshop's template against the job, each item rated and photographed. An Estimate is built from the results with the organisation's currency, sent to the customer as a link, and approved on their phone, line by line, into EstimateApproval. The work is done against the clock (TimeEntry) and signed off (QualityCheck). Completing the job goes through the outbox: passport events are appended for every part fitted and every service done, reservations are consumed into StockMovements, and the customer is told.

An Invoice is raised from the job's lines with the tax rate in force and a gapless number, posted to the ledger as revenue, tax and a receivable. A Payment settles it by one of the methods — cash and card to the bank; a card through the provider posts the same as a card at the counter; loyalty points, a gift card or money held go against their own liabilities — and each posting releases exactly what it should. recordPayment in apps/web/src/lib/money.ts also awards loyalty points, settles a referral if this was the referred customer's first paid bill, and sends a receipt. A CreditNote reverses; a Refund sends money back, or keeps it as credit on the customer's account.

## 6.2 Card payment through a provider



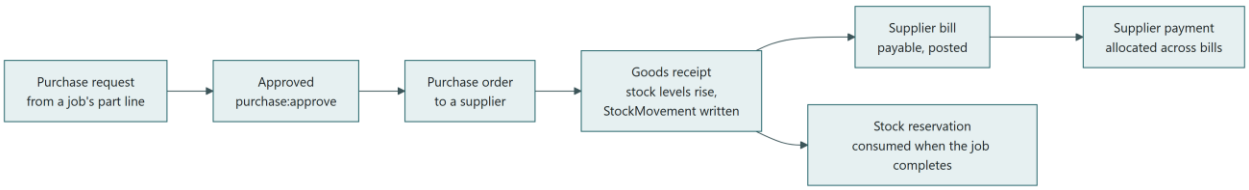

*Pay now, from the portal: the attempt, the provider's checkout, one settlement, and reconciliation.*

A payment provider sits behind a seam of four calls in packages/core (open a checkout, ask what became of it, refund, read a webhook), with a stub that takes pretend money at a checkout page of ours and Stripe written against its HTTP API behind the same seam; PAYMENT_PROVIDER chooses. "Pay now" on an outstanding invoice in the portal opens a PaymentAttempt and sends the customer to the checkout. The provider's answer arrives by webhook or when the customer returns and the page asks; settleAttempt claims the move from pending with a conditional update before recording anything, so two answers record one payment, posted exactly like a card at the counter. A declined card is a row with the provider's reason. A refund by card goes back through the provider, asked before the refund is recorded. Reconciliation asks the provider about every attempt of the last ninety days and names each difference between its record and the books without changing anything; the provider is a witness, never the record.

## 6.3 The counter, gift cards and money held

A counter sale is the same money path without a job: a CounterSale with lines and tenders in one transaction, stock moved as it is sold, the ledger posted per tender, inside a TillSession that is counted at close. A gift card is sold at the counter or on its own page, and the sale posts a liability, not revenue; spending it releases the liability and never touches cash. A refund by gift card goes back onto the card that paid. Money held — a deposit before the work, or a refund kept as credit — is the same idea for a customer rather than a card: a liability until an invoice takes it or it goes back, oldest first, refused rather than clamped. packages/core/src/giftcards and deposits hold the rules; apps/web/src/lib/giftcards.ts, deposits.ts and counter.ts do the work.

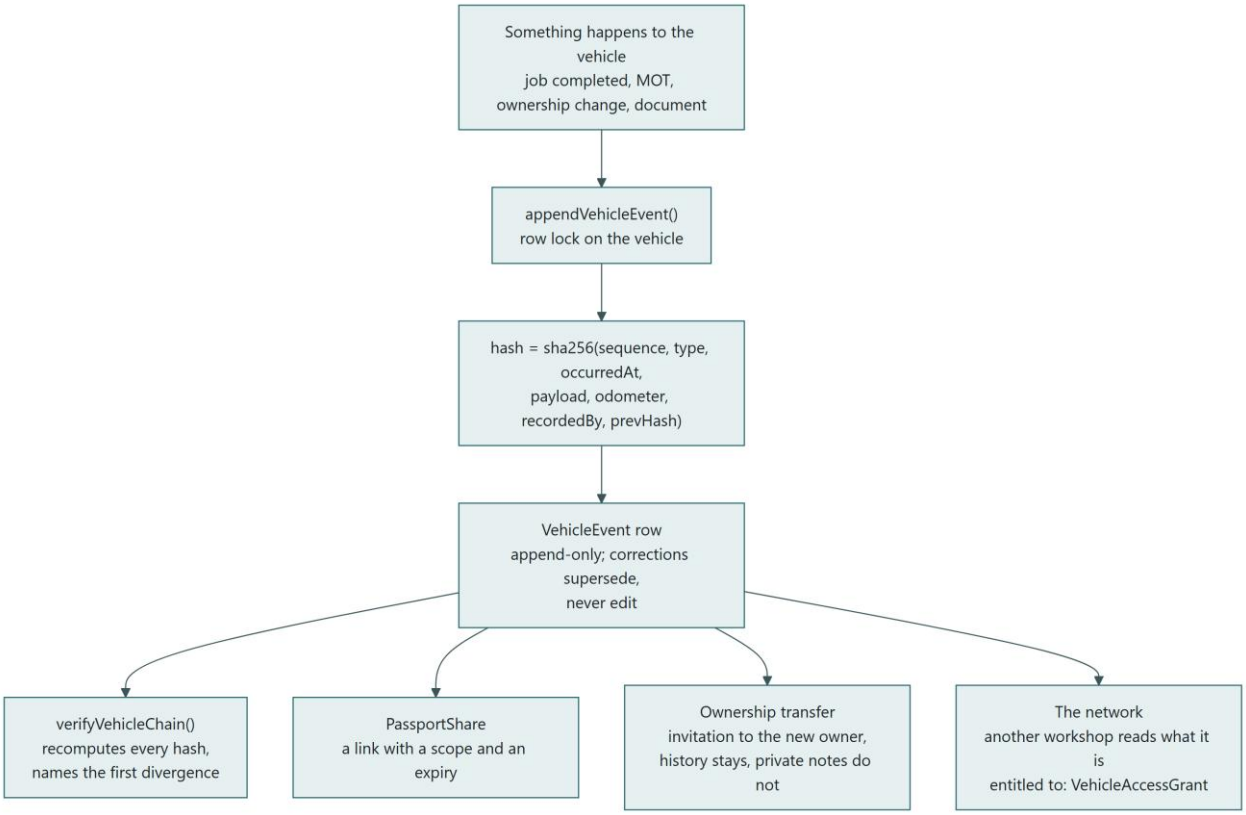
## 6.4 Purchasing and stock




*Request → order → receipt → bill → payment, with stock rising at the receipt.*

A part a job needs and the workshop does not have becomes a PurchaseRequest, approved by a role that may approve spending, gathered onto a PurchaseOrder to a supplier. The GoodsReceipt is the moment stock exists: StockLevel rises, a StockMovement records it with the order as its reason, and the price paid is kept on the line for the product's history. The SupplierBill posts a payable; SupplierPayment settles bills and is allocated across them. Transfers between sites are dispatched at one warehouse and received at another, and the stock is in transit between.

## 6.5 The passport

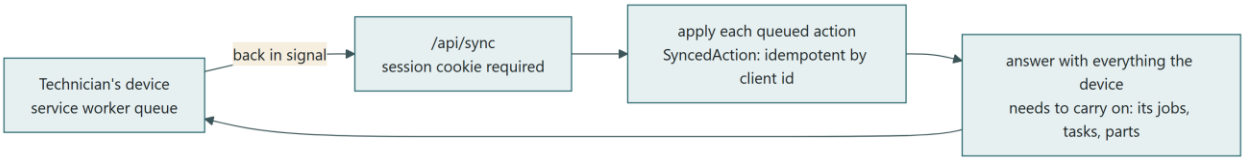



*An event on a vehicle: chained, verified, shared, transferred, read by the network.*

The passport is the product's reason to exist, and the mechanism is small. Every event on a vehicle is a VehicleEvent whose hash covers its own content and the hash of the event before it (packages/core/src/passport/chain.ts). Altering any event changes its hash and breaks every hash after it; verifyVehicleChain recomputes the chain and names the first divergence, and the passport page shows the verdict. It is deliberately not a blockchain (ADR-0005): the question is whether the operator of this database can silently rewrite the past, and a hash chain with periodic anchoring answers it.

A share is a PassportShare: a link with a scope (what kinds of events) and an expiry, every opening recorded. A transfer of ownership is an invitation to the new owner; the history stays, the previous owner's private notes do not. Another workshop reads what it is entitled to through VehicleAccessGrant, earned by working on the car or granted by the owner, and the same entitlement governs /api/v1.

## 6.6 The technician's device, out of signal

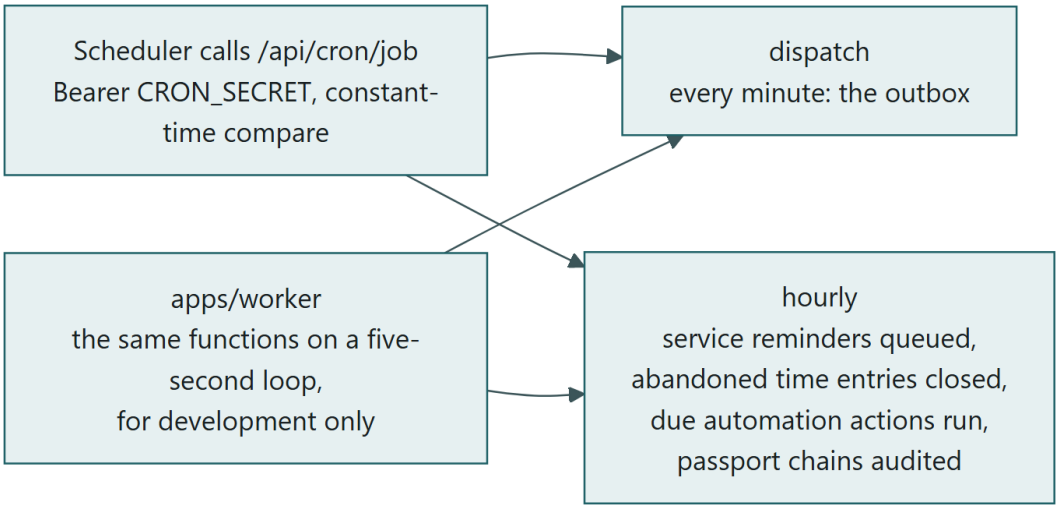



*The queue empties through /api/sync when the device is back in signal.*

The shop floor keeps working without a connection. Actions taken while away — a task ticked, time started, a note — are queued on the device by a service worker and sent to /api/sync when signal returns. The route is authenticated like every other (a queue that accepted work on the strength of an id

in the body would be a write API with no door on it), applies each action once by its client-side id (SyncedAction), and answers with everything the device needs to carry on: its jobs, tasks and parts.

## 6.7 Scheduled work




*Two endpoints, called on a schedule, run the same functions the worker runs on a loop.*

There is no queue server. A scheduler calls /api/cron/dispatch every minute to drain the outbox and /api/cron/hourly to queue service reminders, close abandoned time entries, run due automation actions and audit passport chains. In development apps/worker calls the same functions on a five-second loop. There are not two implementations of any job.

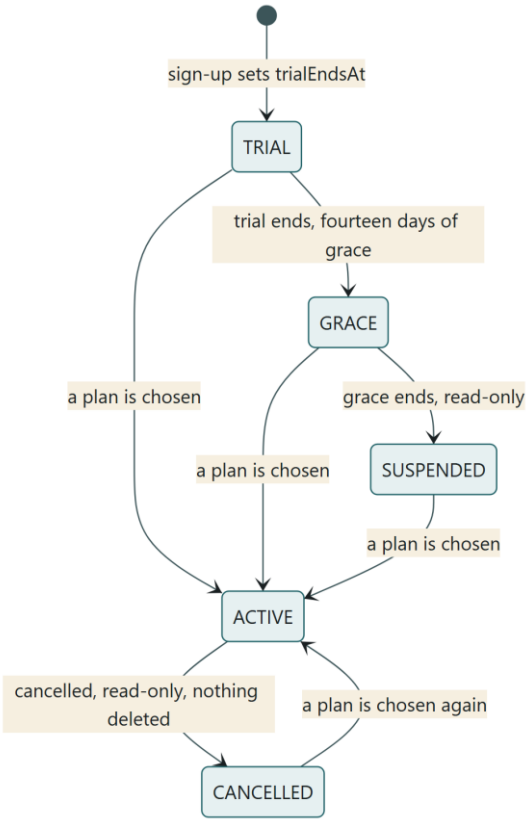
## 6.8 Communication, in and out

Every outbound message is a MessageLog row with a channel and a reason, written before it is sent, then sent through the provider, whose webhooks come back as DeliveryEvents. A reply arrives at /api/inbound as an InboundMessage and is matched to the job it is about, so the conversation is on the job. A bounce or an opt-out writes SuppressedAddress and the product stops sending to it. Automation rules and campaigns produce messages the same way; nothing bypasses the log.

## 6.9 Region and currency

The organisation carries one country, currency, locale and timezone, chosen at sign-up and editable by the owner. The session carries the region and every page formats money and dates from it; invoices and estimates are raised in the organisation's currency; a message to a customer is formatted the same way. A driver in the portal sees each amount in the currency of the workshop that raised it. Changing the currency converts nothing already recorded, and the page says so.

## 6.10 The subscription and the plan




*The subscription's states, read from four dates on the organisation.*

Sign-up sets trialEndsAt. The state is read from the dates each time a session is made and is never stored. A lapsed or cancelled workshop keeps reading and exporting and loses writing; nothing is deleted in any state; choosing a plan reopens it. The plan's limits (sites, bays, vehicles) and features (the ledger, payroll, transfers, the API) are enforced inside the functions that make or do the thing, with the refusal naming the number and the plan.

## 6.11 Answers and the assistant

Ask takes a question in words and answers from the workshop's own data through the same scoped client the pages use, so it cannot see more than the person asking. Every exchange is an AssistantExchange row with the data it was allowed to see. Generated text inside the workflow (a summary for the customer, a reply drafted) is always a draft a person sends, never a decision (ADR-0013).

# 7. Database architecture, domain by domain

One PostgreSQL database, 128 tables in 12 domains, 63 enumerations, 1967 columns, defined in packages/db/prisma/schema.prisma and applied by 37 hand-written migrations. Every table that belongs to a workshop carries orgId, and the tenant scope in packages/db/src/tenant.ts adds it to every query. The tables that are deliberately global — the vehicle and its history, the user, recalls, part fitments — are the ones the network is built on.

## Conventions

- Identifiers are cuids, generated by the application, never sequential, never guessable.
- Every row has createdAt; rows that change have updatedAt; rows that are retired have isActive or a dated column (revokedAt, disabledAt, cancelledAt) rather than being deleted.
- Foreign keys to the organisation cascade; foreign keys between records restrict, so a customer with invoices cannot be deleted by accident.
- Money is Decimal(12,2) with the currency beside it where it can differ; quantities are Decimal where parts are sold by measure.
- Enumerations are Postgres enums, so an invalid status cannot be written; state is read from facts where a stored status could drift.
- Check constraints guard what the application must never do even if it tried: a gift card entry whose sign disagrees with its kind, a passport event out of sequence.
- Unique constraints carry the business rule: one membership per user per organisation, one vehicle per VIN, one supplier name per organisation, one number per document sequence.

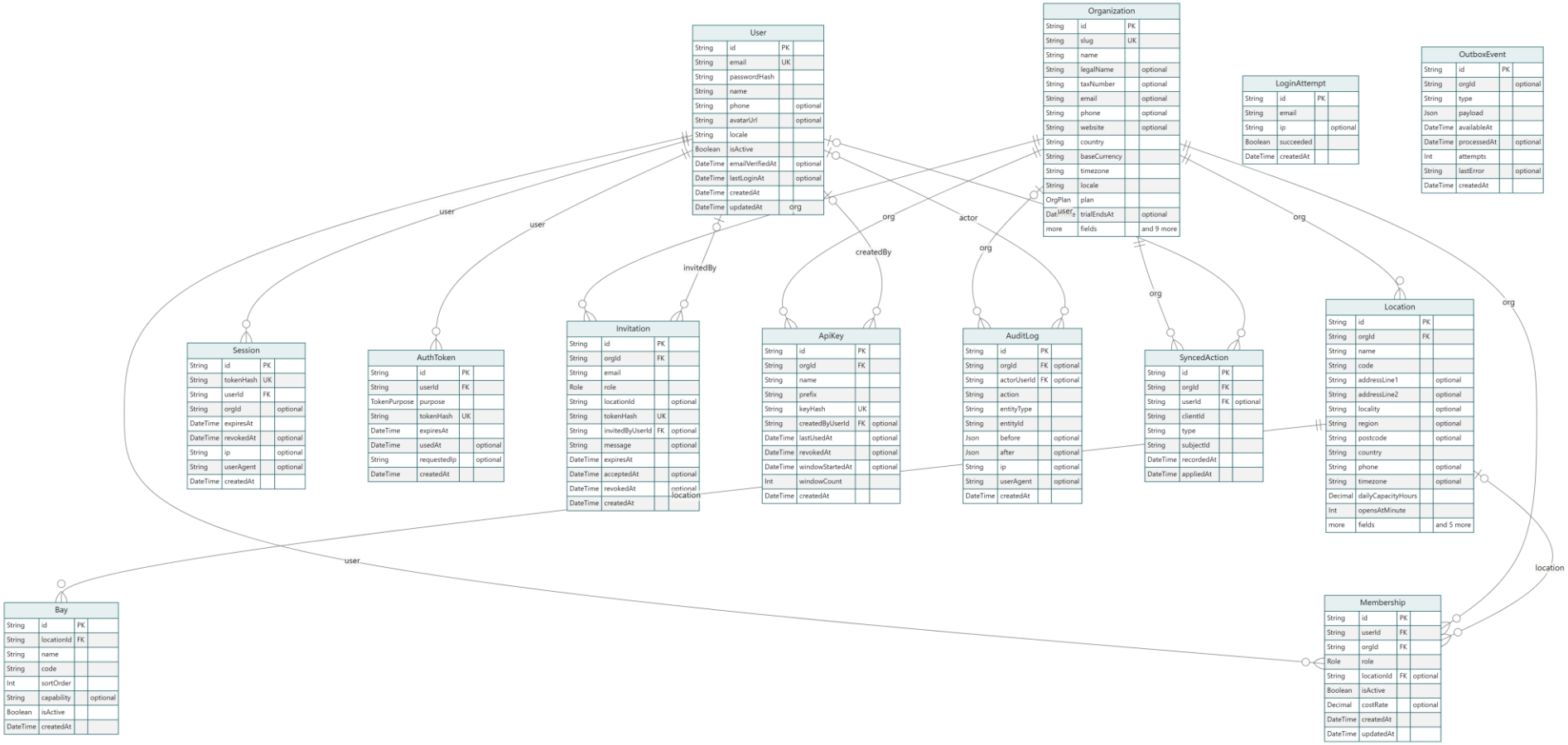
*In the diagrams that follow, PK is the primary key, FK a foreign key, UK a unique column; a column marked optional is nullable. A box that reads "in another domain" is a table drawn in full in another section. Tables with more than fourteen columns show the first fourteen and the count of the rest.*

## 7.1 Tenancy, identity and the audit trail

Everything hangs off the organisation. A workshop is an Organization with one or more Locations, each with Bays; a person is a User with a Membership that carries their role in one organisation. Sessions are rows, not signed tokens: the cookie holds a random token whose SHA-256 digest is the row's key, so a session can be ended on the server and a stolen database does not yield usable cookies (ADR-0006).

The organisation also carries its region (country, currency, locale, timezone) and the four dates its subscription state is read from. AuditLog is written by every consequential action and never rolls the action back if it fails; OutboxEvent is the transactional outbox that carries side effects out of the commit that caused them; SyncedAction is the idempotency record for what a technician's device did while it was out of signal.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Organization` | 23 | global | — |
| `Location` | 19 | organisation | Organization |
| `Bay` | 8 | through Location | Location |
| `User` | 12 | global | — |
| `Membership` | 9 | organisation | User, Organization, Location |
| `Session` | 9 | organisation | User |
| `LoginAttempt` | 5 | global | — |
| `AuthToken` | 8 | global | User |
| `Invitation` | 12 | organisation | Organization, User |
| `ApiKey` | 11 | organisation | Organization, User |
| `AuditLog` | 11 | organisation | Organization, User |
| `OutboxEvent` | 9 | organisation | — |
| `SyncedAction` | 8 | organisation | Organization, User |



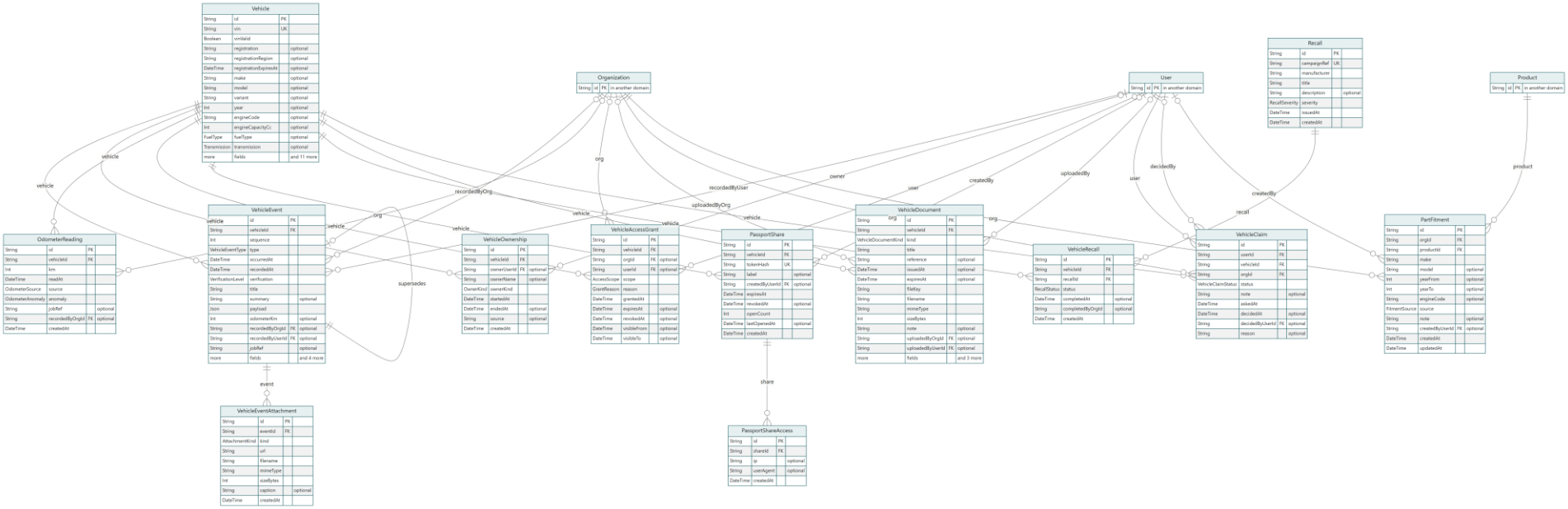

*Entity-relationship diagram: tenancy, identity and the audit trail. Read from schema.prisma.*

## 7.2 The vehicle and its passport

The vehicle is global, not owned by a workshop (ADR-0004): one Vehicle row per VIN, whoever touches it. Its history is VehicleEvent, append-only and hash-chained: each event's hash covers its own content and the hash of its predecessor, so altering any event breaks every hash after it, and a correction is a new event that supersedes an old one rather than an edit. VehicleOwnership is the chain of custody; OdometerReading is kept separately so a reading can be checked against every other.

Who may see what is VehicleAccessGrant (a workshop's entitlement, earned by working on the car or granted by the owner) and PassportShare (a link with a scope and an expiry, every opening recorded in PassportShareAccess). VehicleDocument is the paperwork; VehicleClaim is a warranty or insurance claim against the record; Recall and VehicleRecall are manufacturer notices matched to vehicles; PartFitment is what fits what.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Vehicle` | 25 | global | — |
| `VehicleEvent` | 18 | global | Vehicle, Organization, User, VehicleEvent |
| `VehicleEventAttachment` | 9 | global | VehicleEvent |
| `VehicleOwnership` | 9 | global | Vehicle, User |
| `OdometerReading` | 9 | global | Vehicle, Organization |
| `VehicleAccessGrant` | 11 | organisation | Vehicle, Organization, User |
| `PassportShare` | 10 | global | Vehicle, User |
| `PassportShareAccess` | 5 | global | PassportShare |
| `Recall` | 8 | global | — |
| `VehicleRecall` | 7 | global | Vehicle, Recall |
| `VehicleDocument` | 17 | global | Vehicle, Organization, User |
| `VehicleClaim` | 10 | organisation | User, Vehicle, Organization |
| `PartFitment` | 13 | organisation | Organization, Product, User |



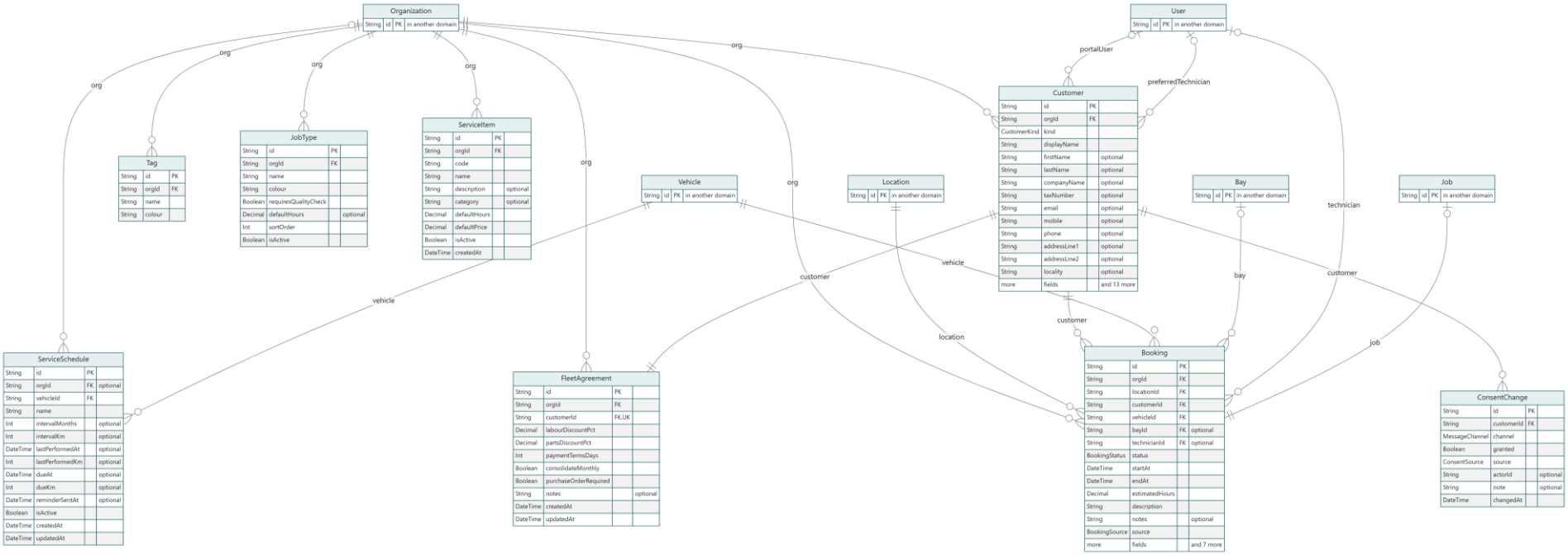

*Entity-relationship diagram: the vehicle and its passport. Read from schema.prisma.*

## 7.3 Customers, bookings and the diary

A Customer belongs to one organisation and may be an individual or a company; a fleet is a company Customer with a FleetAgreement that sets the terms for every car on it. Booking is a diary slot against a bay and a capacity check; JobType and ServiceItem are the workshop's own catalogue of what it does and what it charges; ServiceSchedule is the next service the reminder job will write to the customer about.

ConsentChange is the record of every change to what a customer has agreed to be sent, which is what lets the product answer a complaint with a date rather than an apology.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Customer` | 27 | organisation | Organization, User |
| `Tag` | 4 | organisation | Organization |
| `Booking` | 21 | organisation | Organization, Location, Customer, Vehicle, Bay, User, Job |
| `JobType` | 8 | organisation | Organization |
| `ServiceItem` | 10 | organisation | Organization |
| `ServiceSchedule` | 14 | organisation | Organization, Vehicle |
| `FleetAgreement` | 11 | organisation | Organization, Customer |
| `ConsentChange` | 8 | through Customer | Customer |



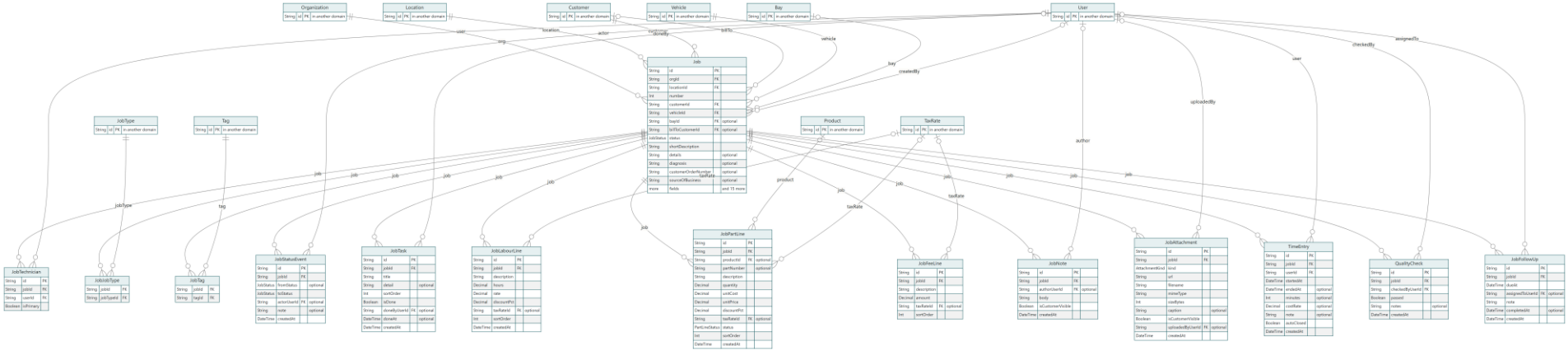

*Entity-relationship diagram: customers, bookings and the diary. Read from schema.prisma.*

## 7.4 The job

The Job is the central record of the workshop. It carries its own lines (labour, parts, fees) rather than pointing at an estimate's, because what was quoted and what was done are allowed to differ and both are kept. Every status change is a JobStatusEvent, so the job's timeline is a query, not a memory. TimeEntry is the technician's clock; QualityCheck is the sign-off before the customer is told; JobFollowUp is the call afterwards.

Totals are never stored on the job: they are derived from the lines each time they are read (ADR-0011), which is why a line changing cannot leave a stale figure behind.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Job` | 29 | organisation | Organization, Location, Customer, Vehicle, Bay, User |
| `JobTechnician` | 4 | through Job | Job, User |
| `JobJobType` | 2 | through Job | Job, JobType |
| `JobTag` | 2 | through Job | Job, Tag |
| `JobStatusEvent` | 7 | through Job | Job, User |
| `JobTask` | 9 | through Job | Job, User |
| `JobLabourLine` | 9 | through Job | Job, TaxRate |
| `JobPartLine` | 13 | through Job | Job, Product, TaxRate |
| `JobFeeLine` | 6 | through Job | Job, TaxRate |
| `JobNote` | 6 | through Job | Job, User |
| `JobAttachment` | 11 | through Job | Job, User |
| `TimeEntry` | 10 | through Job | Job, User |
| `QualityCheck` | 6 | through Job | Job, User |
| `JobFollowUp` | 7 | through Job | Job, User |



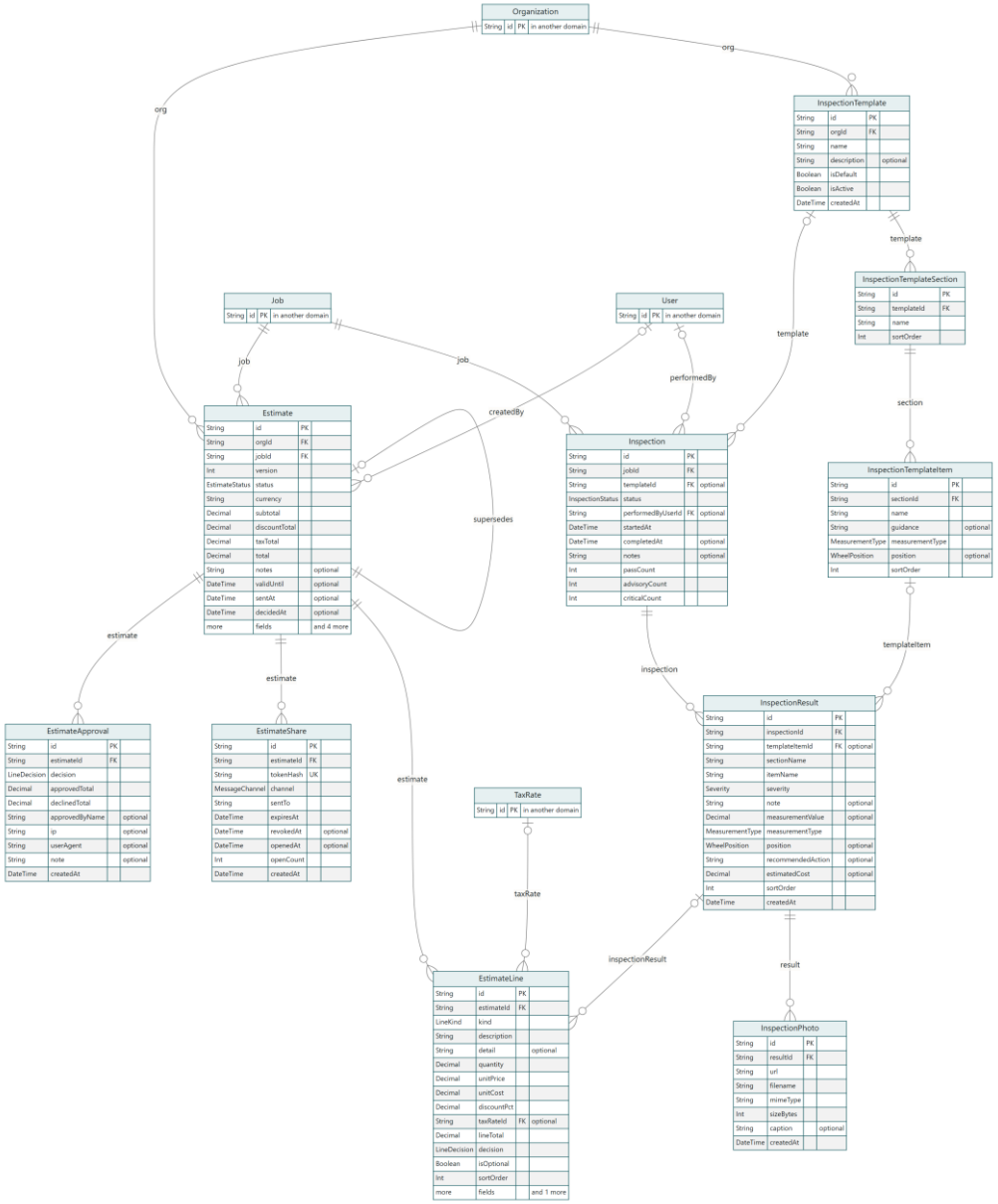
*Entity-relationship diagram: the job. Read from schema.prisma.*

## 7.5 Inspections and estimates

An InspectionTemplate is the workshop's own checklist, in sections and items; an Inspection is one run of it against one job, each item answered in an InspectionResult with a rating and, where it matters, an InspectionPhoto. The photograph is the trust mechanism (ADR-0012): a customer approves brake work because they can see the disc, not because they are told about it.

An Estimate is built from the results, line by line, with the currency of the organisation on it. It goes to the customer as an EstimateShare, a link with a token, and comes back as an EstimateApproval that records who approved which lines, when, from which address. An approved estimate is what a job's work is authorised against.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `InspectionTemplate` | 7 | organisation | Organization |
| `InspectionTemplateSection` | 4 | through InspectionTemplate | InspectionTemplate |
| `InspectionTemplateItem` | 7 | through InspectionTemplateSection | InspectionTemplateSection |
| `Inspection` | 11 | through Job | Job, InspectionTemplate, User |
| `InspectionResult` | 14 | through Inspection | Inspection, InspectionTemplateItem |
| `InspectionPhoto` | 8 | through InspectionResult | InspectionResult |
| `Estimate` | 18 | organisation | Organization, Job, User, Estimate |
| `EstimateLine` | 15 | through Estimate | Estimate, TaxRate, InspectionResult |
| `EstimateShare` | 10 | through Estimate | Estimate |
| `EstimateApproval` | 10 | through Estimate | Estimate |



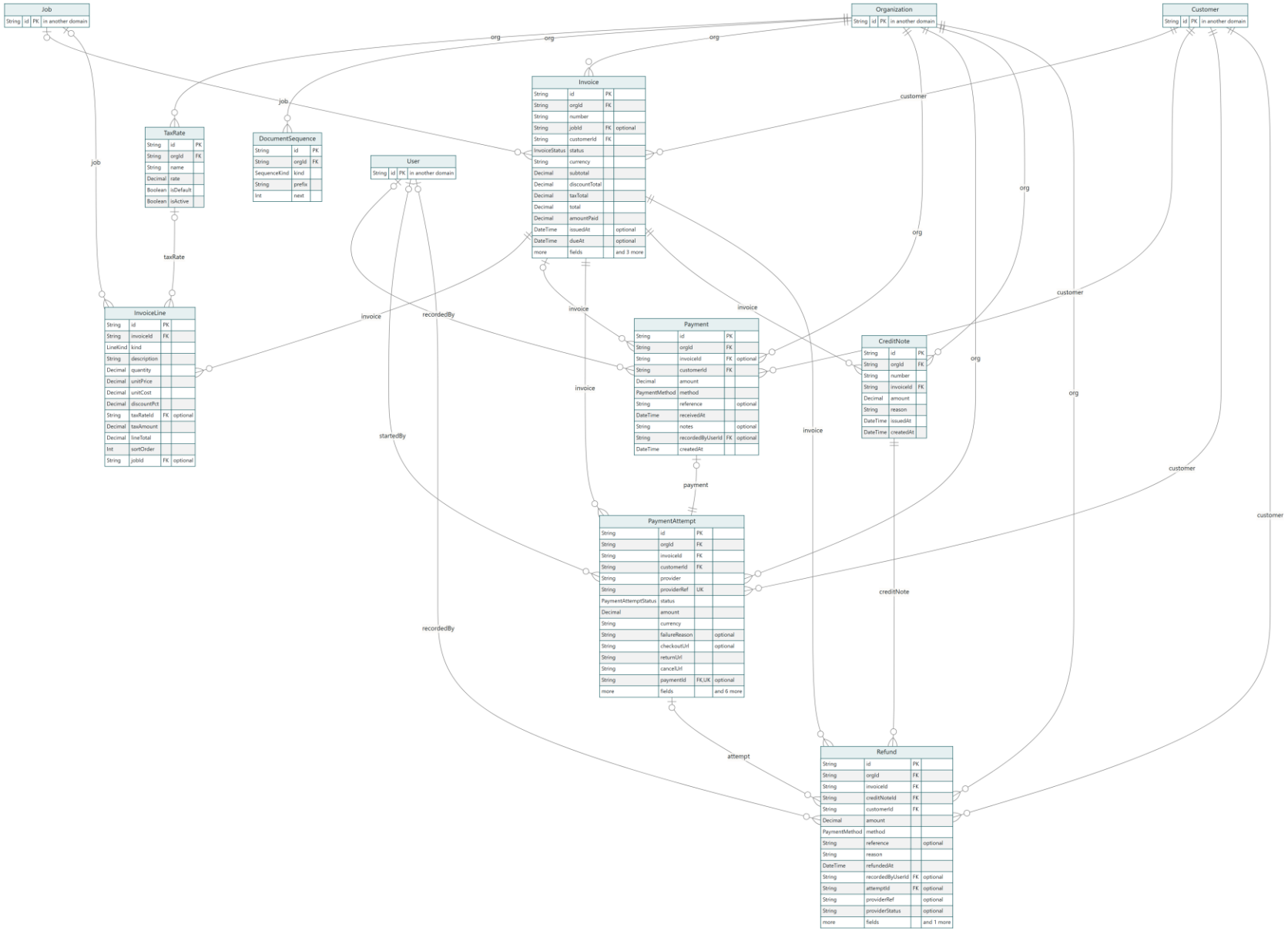

*Entity-relationship diagram: inspections and estimates. Read from schema.prisma.*

## 7.6 Invoices, payments and refunds

An Invoice is raised from a job with its lines copied and its tax computed by the TaxRate in force; its number comes from DocumentSequence, which hands out gapless numbers per organisation and per document kind under a row lock. Totals are Decimal(12,2) in the database and integer minor units in the rules, converted at the boundary, so arithmetic never happens in floating point.

A Payment is one tender against one invoice, with a method that decides its ledger account: cash and card to the bank, points to the loyalty liability, a gift card to the gift card liability, money held to the deposit liability. A CreditNote reverses an invoice in whole or part; a Refund is money going back out, with a method of its own, and may instead be kept as credit. A PaymentAttempt is one try at paying through the provider, moving once from pending; a succeeded attempt produces exactly one Payment, and a Refund sent back through the provider names the attempt it went back to.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `TaxRate` | 6 | organisation | Organization |
| `DocumentSequence` | 5 | organisation | Organization |
| `Invoice` | 17 | organisation | Organization, Job, Customer |
| `InvoiceLine` | 13 | through Invoice | Invoice, TaxRate, Job |
| `Payment` | 11 | organisation | Organization, Invoice, Customer, User |
| `PaymentAttempt` | 20 | organisation | Organization, Invoice, Customer, Payment, User |
| `CreditNote` | 8 | organisation | Organization, Invoice |
| `Refund` | 15 | organisation | Organization, Invoice, CreditNote, Customer, User, PaymentAttempt |



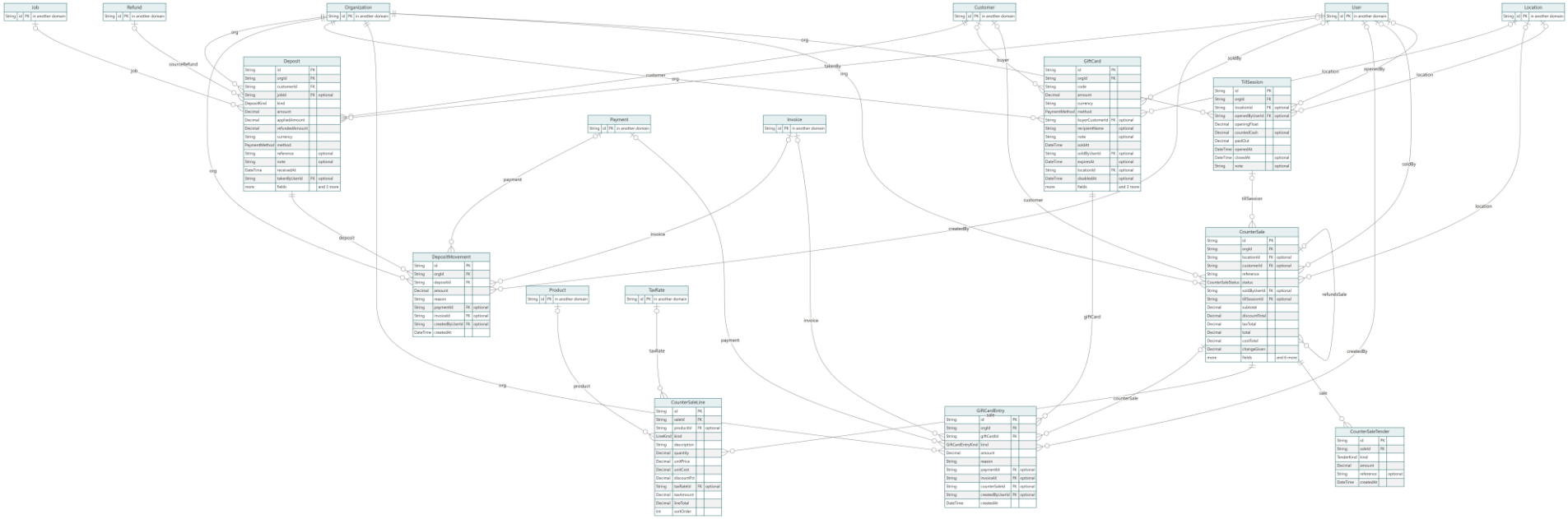

*Entity-relationship diagram: invoices, payments and refunds. Read from schema.prisma.*

## 7.7 The counter, gift cards and money held

The counter is a sale without a job: a CounterSale with its lines and one or more CounterSaleTenders, inside a TillSession that is opened, counted and closed. A gift card is a liability the workshop sells: GiftCard is the card and GiftCardEntry every movement on it, typed as an issue, a redemption, a refund or an adjustment, with a database constraint that refuses a kind whose sign disagrees with it. The card's status is never stored; it is read from its entries and dates.

Money held ahead of a bill is a Deposit of kind deposit or credit, and DepositMovement is every amount that came in, went against a bill, or went back. What is held is the sum, never a stored figure.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Deposit` | 17 | organisation | Organization, Customer, Job, User, Refund |
| `DepositMovement` | 9 | organisation | Organization, Deposit, Payment, Invoice, User |
| `GiftCard` | 17 | organisation | Organization, Customer, User, Location |
| `GiftCardEntry` | 11 | organisation | Organization, GiftCard, Payment, Invoice, CounterSale, User |
| `CounterSale` | 20 | organisation | Organization, Location, Customer, User, TillSession, CounterSale |
| `CounterSaleLine` | 13 | through CounterSale | CounterSale, Product, TaxRate |
| `CounterSaleTender` | 6 | through CounterSale | CounterSale |
| `TillSession` | 10 | organisation | Organization, Location, User |



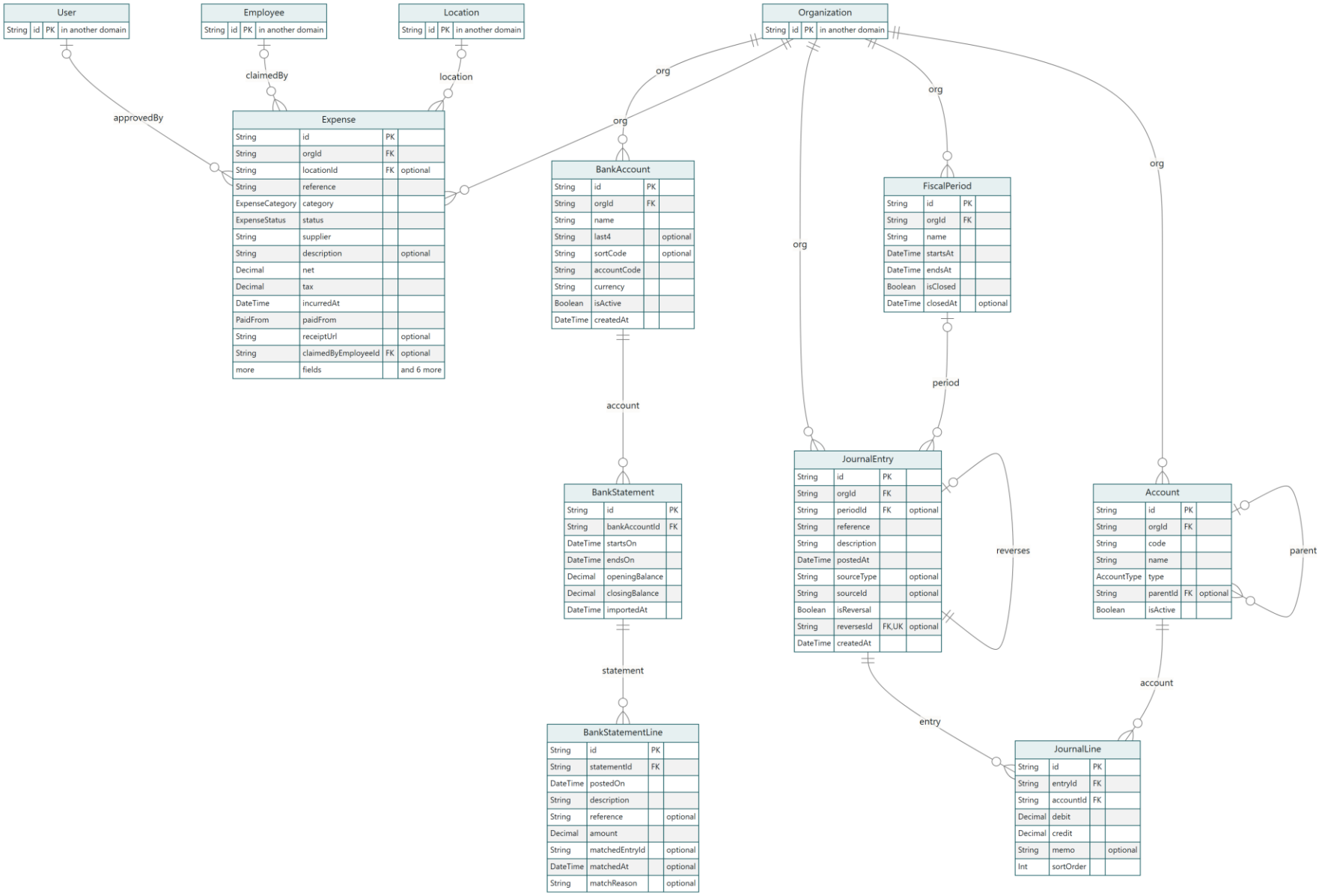

*Entity-relationship diagram: the counter, gift cards and money held. Read from schema.prisma.*

## 7.8 The ledger, expenses and banking

The ledger is double entry. Account is the chart, filled to a standard set for every organisation; JournalEntry is one balanced posting with a source type and id pointing back at the invoice, payment, expense or bill that caused it, and JournalLine is each debit or credit against an account. A posting that does not balance is refused before it is written. A reversal is a new entry that points at the one it reverses.

FiscalPeriod is what a bookkeeper closes; a closed period refuses postings. Expense is money spent outside a supplier bill, claimed and approved; BankAccount, BankStatement and BankStatementLine are the imported bank feed and its matching against ledger entries, line by line, with the reason recorded.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Account` | 7 | organisation | Organization, Account |
| `FiscalPeriod` | 7 | organisation | Organization |
| `JournalEntry` | 11 | organisation | Organization, FiscalPeriod, JournalEntry |
| `JournalLine` | 7 | through JournalEntry | JournalEntry, Account |
| `Expense` | 20 | organisation | Organization, Location, Employee, User |
| `BankAccount` | 9 | organisation | Organization |
| `BankStatement` | 7 | through BankAccount | BankAccount |
| `BankStatementLine` | 9 | through BankStatement | BankStatement |



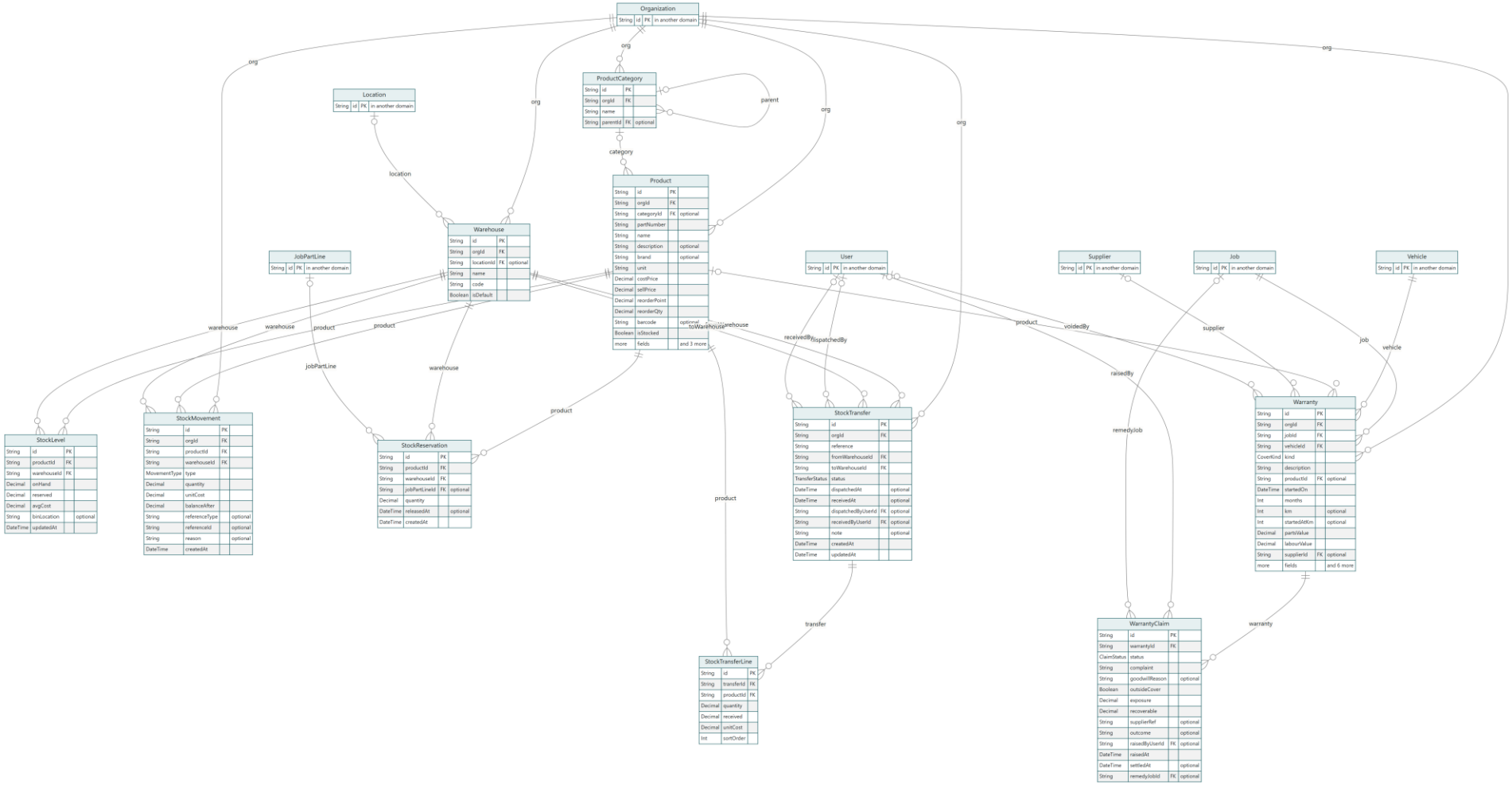

*Entity-relationship diagram: the ledger, expenses and banking. Read from schema.prisma.*

## 7.9 Parts and stock

Product is a part or a stocked item with a category; Warehouse is where stock sits (a site or a van); StockLevel is the quantity on hand per product per warehouse, and StockMovement is every change to it with a reason. StockReservation holds parts for a job from the moment they are on it, and is consumed when the job completes, so two jobs cannot both count the same filter.

StockTransfer moves stock between warehouses, dispatched at one and received at the other. Warranty and WarrantyClaim are the cover on a part or a job and the claim against it when it fails.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `ProductCategory` | 4 | organisation | Organization, ProductCategory |
| `Product` | 17 | organisation | Organization, ProductCategory |
| `Warehouse` | 6 | organisation | Organization, Location |
| `StockLevel` | 8 | through Product | Product, Warehouse |
| `StockMovement` | 12 | organisation | Organization, Product, Warehouse |
| `StockReservation` | 7 | through Product | Product, Warehouse, JobPartLine |
| `StockTransfer` | 13 | organisation | Organization, Warehouse, User |
| `StockTransferLine` | 7 | through StockTransfer | StockTransfer, Product |
| `Warranty` | 20 | organisation | Organization, Job, Vehicle, Product, Supplier, User |
| `WarrantyClaim` | 14 | through Warranty | Warranty, User, Job |



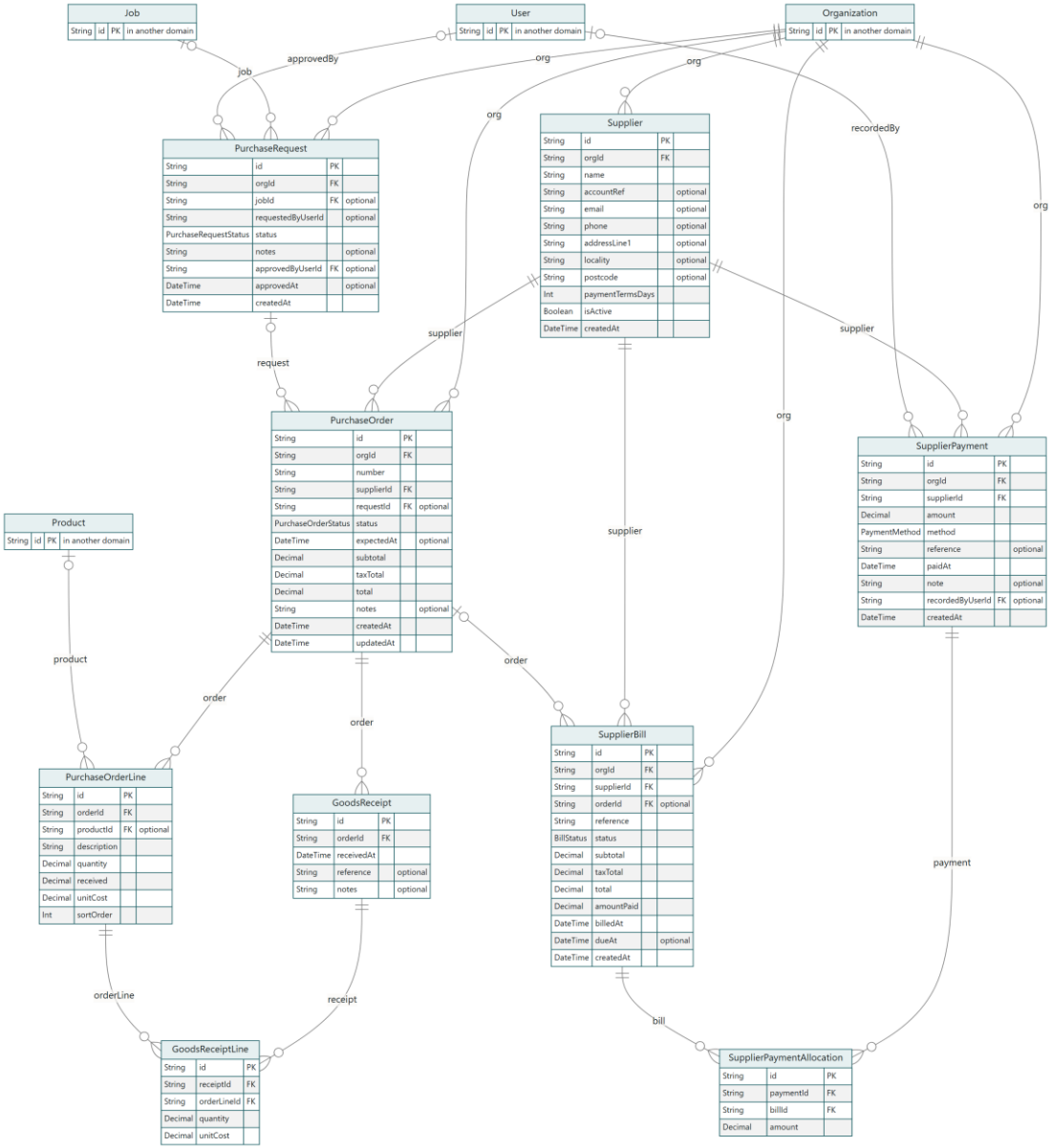

*Entity-relationship diagram: parts and stock. Read from schema.prisma.*

## 7.10 Suppliers and purchasing

A PurchaseRequest is raised from a job's part line by whoever needs it and approved by whoever may approve spending; approved requests become a PurchaseOrder to a Supplier, line by line. GoodsReceipt is what actually arrived, which raises stock levels and writes movements; SupplierBill is what the supplier then asks for, posted as a payable; SupplierPayment settles bills, allocated across them in SupplierPaymentAllocation.

Price history lives on the order lines, so the product page can say what this part cost last time and from whom.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Supplier` | 12 | organisation | Organization |
| `PurchaseRequest` | 9 | organisation | Organization, Job, User |
| `PurchaseOrder` | 13 | organisation | Organization, Supplier, PurchaseRequest |
| `PurchaseOrderLine` | 8 | through PurchaseOrder | PurchaseOrder, Product |
| `GoodsReceipt` | 5 | through PurchaseOrder | PurchaseOrder |
| `GoodsReceiptLine` | 5 | through GoodsReceipt | GoodsReceipt, PurchaseOrderLine |
| `SupplierBill` | 13 | organisation | Organization, Supplier, PurchaseOrder |
| `SupplierPayment` | 10 | organisation | Organization, Supplier, User |
| `SupplierPaymentAllocation` | 4 | through SupplierPayment | SupplierPayment, SupplierBill |



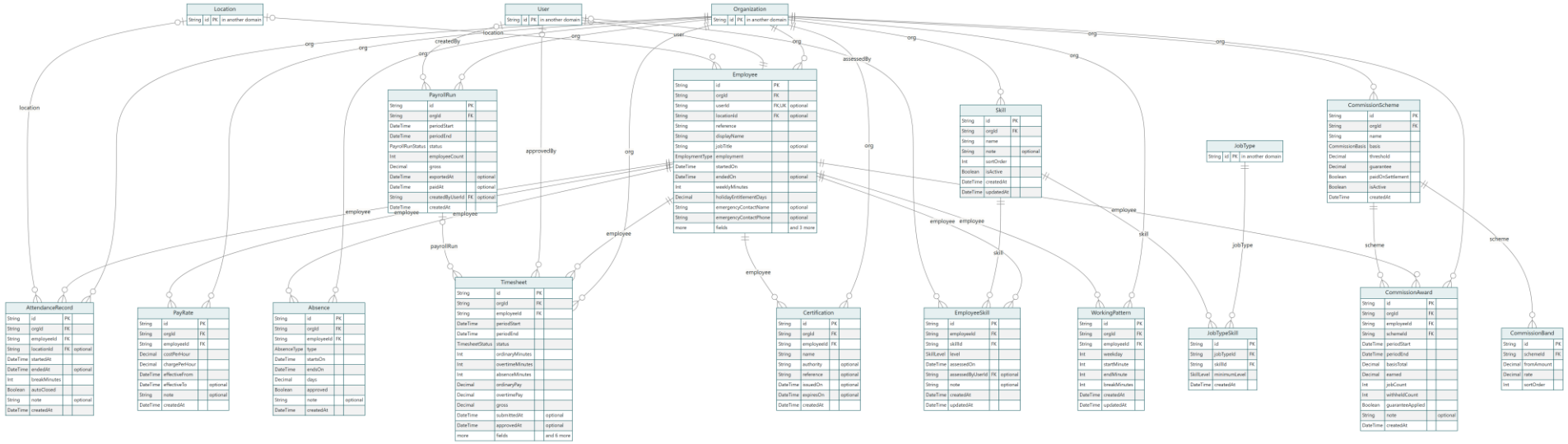

*Entity-relationship diagram: suppliers and purchasing. Read from schema.prisma.*

## 7.11 People, skills and payroll

Employee is a person on the payroll, which is not the same as a User who can sign in: a technician may be both, an owner may be neither. PayRate is what they are paid and from when; AttendanceRecord, Timesheet and Absence are the hours; Skill, EmployeeSkill, JobTypeSkill and Certification are what they can do and what a job needs, which the planner uses to refuse a job to a bay that cannot take it. WorkingPattern is the week they work.

PayrollRun is one pay period, computed and approved by a role that is deliberately separate from operational reporting, because the manager who runs the productivity board must not see what the person beside them is paid. CommissionScheme, CommissionBand and CommissionAward are earnings on work sold.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `Employee` | 17 | organisation | Organization, User, Location |
| `PayRate` | 9 | organisation | Organization, Employee |
| `AttendanceRecord` | 10 | organisation | Organization, Employee, Location |
| `Timesheet` | 20 | organisation | Organization, Employee, User, PayrollRun |
| `Absence` | 10 | organisation | Organization, Employee |
| `Certification` | 9 | organisation | Organization, Employee |
| `Skill` | 8 | organisation | Organization |
| `EmployeeSkill` | 9 | through Employee | Employee, Skill, User |
| `JobTypeSkill` | 5 | through JobType | JobType, Skill |
| `WorkingPattern` | 9 | organisation | Organization, Employee |
| `PayrollRun` | 11 | organisation | Organization, User |
| `CommissionScheme` | 9 | organisation | Organization |
| `CommissionBand` | 5 | through CommissionScheme | CommissionScheme |
| `CommissionAward` | 13 | organisation | Organization, Employee, CommissionScheme |



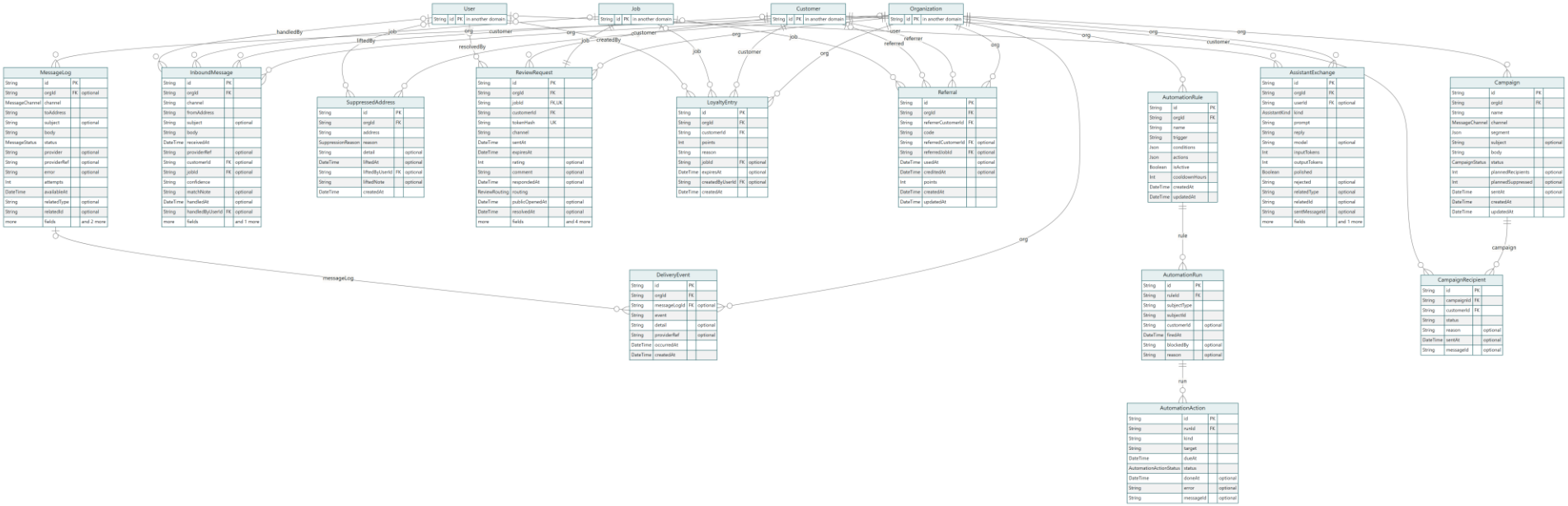

*Entity-relationship diagram: people, skills and payroll. Read from schema.prisma.*

## 7.12 Communication, retention and automation

Everything sent to a customer is a MessageLog row before it is a message: the channel, the body, the reason, and then a DeliveryEvent for every state the provider reports back. InboundMessage is the reply, matched to the job it is about; SuppressedAddress is where the product stops sending because something bounced or somebody asked.

Retention is ReviewRequest (a strong rating invited to a public review, a poor one into a private conversation), LoyaltyEntry (points earned and spent, mirrored in the ledger as a liability) and Referral (a code, the person it brought, and the reward settled when their first bill is paid). AutomationRule, AutomationRun and AutomationAction are the workshop's own rules and every time they fired; Campaign and CampaignRecipient are one-off sends to a segment. AssistantExchange is every question asked of the assistant and the answer given, with the data it was allowed to see.

| Table | Columns | Tied to a workshop | Points at |
|---|---|---|---|
| `MessageLog` | 16 | organisation | Organization |
| `InboundMessage` | 15 | organisation | Organization, Customer, Job, User |
| `SuppressedAddress` | 9 | organisation | Organization, User |
| `DeliveryEvent` | 8 | organisation | Organization, MessageLog |
| `ReviewRequest` | 18 | organisation | Organization, Job, Customer, User |
| `LoyaltyEntry` | 9 | organisation | Organization, Customer, Job, User |
| `Referral` | 11 | organisation | Organization, Customer, Job |
| `AutomationRule` | 10 | organisation | Organization |
| `AutomationRun` | 8 | through AutomationRule | AutomationRule |
| `AutomationAction` | 9 | through AutomationRun | AutomationRun |
| `Campaign` | 13 | organisation | Organization |
| `CampaignRecipient` | 7 | through Campaign | Campaign, Customer |
| `AssistantExchange` | 15 | organisation | Organization, User |




*Entity-relationship diagram: communication, retention and automation. Read from schema.prisma.*

# 8. Security in the data path

docs/02-design/05-security-model.md is the full model; this is where it touches the data path.

- Authentication is a row per session, found by the digest of the cookie, expiring, revocable. Passwords are hashed with a modern KDF; LoginAttempt rate-limits by address and by account; AuthToken carries the one-time links for sign-up confirmation, invitations and password resets.
- Authorisation is three layers, all of which must pass: the permission (deny by default), the scope (the row must be in the caller's tenant, or entitled by ownership or grant), and the plan (the feature or the room for one more). A cross-tenant request by id is not found.
- Tenant isolation is in the data layer (ADR-0010): the scoped client is the only one a page receives. Its known gap is stated: update and delete by id are not narrowed, so the convention is to load through the scoped client first, and row-level security in Postgres is the intended successor.
- Machine routes fail closed: /api/cron refuses when no secret is configured; /api/v1 keys are hashed at rest; /api/sync needs the same cookie as a page.
- Public links carry their own scope and expiry and record every opening; the passport exception (a workshop reading a car it did not own) is an explicit grant, never an absence of a check.
- The response carries a content security policy with a per-request nonce and the usual headers, checked by verify-headers.ts on every surface.
- Authorisation is tried, not trusted: verify-authz.ts opens every workshop page as every role and reads every server action for its permission before each phase ships.

# 9. The user interface: wireframes and photographs

The wireframes are the intent, drawn before the screens were built and kept in docs/02-design/09-wireframes.md. The photographs are the built screens, taken from the running application by infra/scripts/design-shots.ts on the day this document was built. The design book (docs/Carnet-Design-Book.pdf) explains each design; this section shows what the system looks like while it does what the sections above describe.

## 9.1 Wireframes

THE WORKSHOP SHELL: GROUPS NAMED FOR WHAT A PERSON IS DOING; ITEMS A ROLE CANNOT REACH ARE ABSENT.

```
+---------------+-------------------------------------------------------+
| Hillside      |  WEDNESDAY, 9 SEPTEMBER 2026                          |
| Carnet Workshop|  Good afternoon, Ahmed         [Open diary] [New job]     |
+---------------+-------------------------------------------------------+
| WORKSHOP      |                                                       |
|  Dashboard  * |                     MAIN CONTENT                      |
|  Diary        |                                                       |
|  Jobs      [1]|                                                       |
|  Customers    |                                                       |
|  Vehicles     |                                                       |
| MONEY         |                                                       |
|  Invoices     |                                                       |
|  Payments     |                                                       |
|  Reports      |                                                       |
| INVENTORY     |                                                       |
| FINANCE       |                                                       |
| ADMIN         |                                                       |
+---------------+                                                       |
| Ahmed Bakri   |                                                       |
| Owner         |                                                       |
| Sign out      |                                                       |
+---------------+-------------------------------------------------------+
```

### THE JOB: THE CENTRAL SCREEN.

```
JOB #10482 - HILLSIDE ROAD
BMW X5  [MK20 XRT]  (*) Customer approval        [Vehicle passport]
Brake system repair

+-- WAITING ON THE CUSTOMER ------------------------------------------------+
| Work cannot proceed until the estimate is approved. Sent 9 Sept, 10:42.   |
+---------------------------------------------------------------------------+

+--------------------------------------------+  +------------------------+
| VEHICLE AND CUSTOMER                       |  | PROGRESS               |
| Customer  Maria Okonkwo   Contact 07700... |  |  (v) Vehicle received  |
| Vehicle   2020 BMW X5     VIN WBA JA9C50... |  |  (v) Inspection        |
| Technician John Adeyemi   Bay  Bay 1       |  |  (v) Diagnosis         |
| Promised  17:00           Clocked 0.8h     |  |  (v) Estimate created  |
| DIAGNOSIS                                  |  |  (o) Customer approval |
| Front brake pads worn beyond the wear...   |  |  ( ) Approved          |
+--------------------------------------------+  |  ( ) Repair            |
                                                |  ( ) Quality control   |
+----------------------------[x2] [!3] [v5]-+  |  ( ) Completed         |
| INSPECTION            John Adeyemi         |  |  ( ) Invoice           |
| SECTION  ITEM         MEAS.  FINDING  EST. |  |  ( ) Payment           |
| Brakes   Front pads   1.5mm  Critical £420 |  +------------------------+
| Brakes   Front discs  25.4mm Critical £410 |
| Tyres    Front left   2.8mm  Advisory £165 |  +------------------------+
| Bonnet   Battery      11.9V  Advisory £240 |  | ESTIMATE v1     [sent] |
+--------------------------------------------+  | Subtotal    £1,257.50  |
                                                | Tax           £251.50  |
+--------------------------------------------+  | Total       £1,509.00  |
| LABOUR AND PARTS                           |  +------------------------+
| Charged at book time; costed at clocked time|
| Brake inspection      1.00h  £95   £95.00  |  +------------------------+
| Brake replacement     3.50h  £95  £332.50  |  | HISTORY                |
| BMW front pads    34116794300 1  £420.00   |  | Customer approval      |
| BMW front discs   34116864059 1  £410.00   |  | Sara Whitfield 10:42   |
|                          TOTAL  £1,509.00  |  | Estimate created 09:50 |
+--------------------------------------------+  +------------------------+
```

### THE PASSPORT, AS THE WORKSHOP SEES IT.

```
VEHICLE PASSPORT
2020 BMW X5  [MK20 XRT]
xDrive30d M Sport                          [Share history]  [New job]

+----------------------------------------+  +-------------------------+
| IDENTITY          (v) History intact   |  | OPEN RECALLS            |
|                       Chain E5A9-A764  |  | EGR cooler coolant leak |
| VIN         WBA JA9C50 9G000000        |  | BMW-0072340100          |
| Registration MK20 XRT   Engine B57D30A |  +-------------------------+
| Odometer    82,430 km   Avg 889 km/month |  | NEXT SERVICE            |
| Current owner Maria Okonkwo            |  | Due at    94,150 km     |
| Owners on record 3   Events on record 7 |  | Remaining 11,720 km     |
+----------------------------------------+  | Projected Mar 2026      |
                                             | (*) Overdue             |
+----------------------------------------+  +-------------------------+
| HISTORY           7 entries            |  | JOBS AT THIS WORKSHOP   |
| 2025                                   |  | #10482  Customer appr.  |
|  o TIRES  19 Mar 2025  78,150 km       |  +-------------------------+
|    Two front tyres replaced            |  | OWNERSHIP               |
|    Michelin Pilot Sport 4 SUV 265/50 R19 |  | Maria Okonkwo           |
|    [Workshop] Kirkstall Autocentre     |  | 02/07/2025 - present    |
|                                        |  | Peter Vasquez           |
|  o OWNERSHIP 2 Jul 2025  74,800 km     |  | 18/05/2023 - 02/07/2025 |
|    Ownership transferred               |  | Northgate Leasing Ltd   |
|    From Peter Vasquez to Maria Okonkwo |  | 03/03/2020 - 18/05/2023 |
|    [Owner]                             |  +-------------------------+
| 2024                                   |
|  o RECALL 4 Oct 2024                   |
|    Safety recall issued: EGR cooler    |
|    [Official]                          |
+----------------------------------------+
```

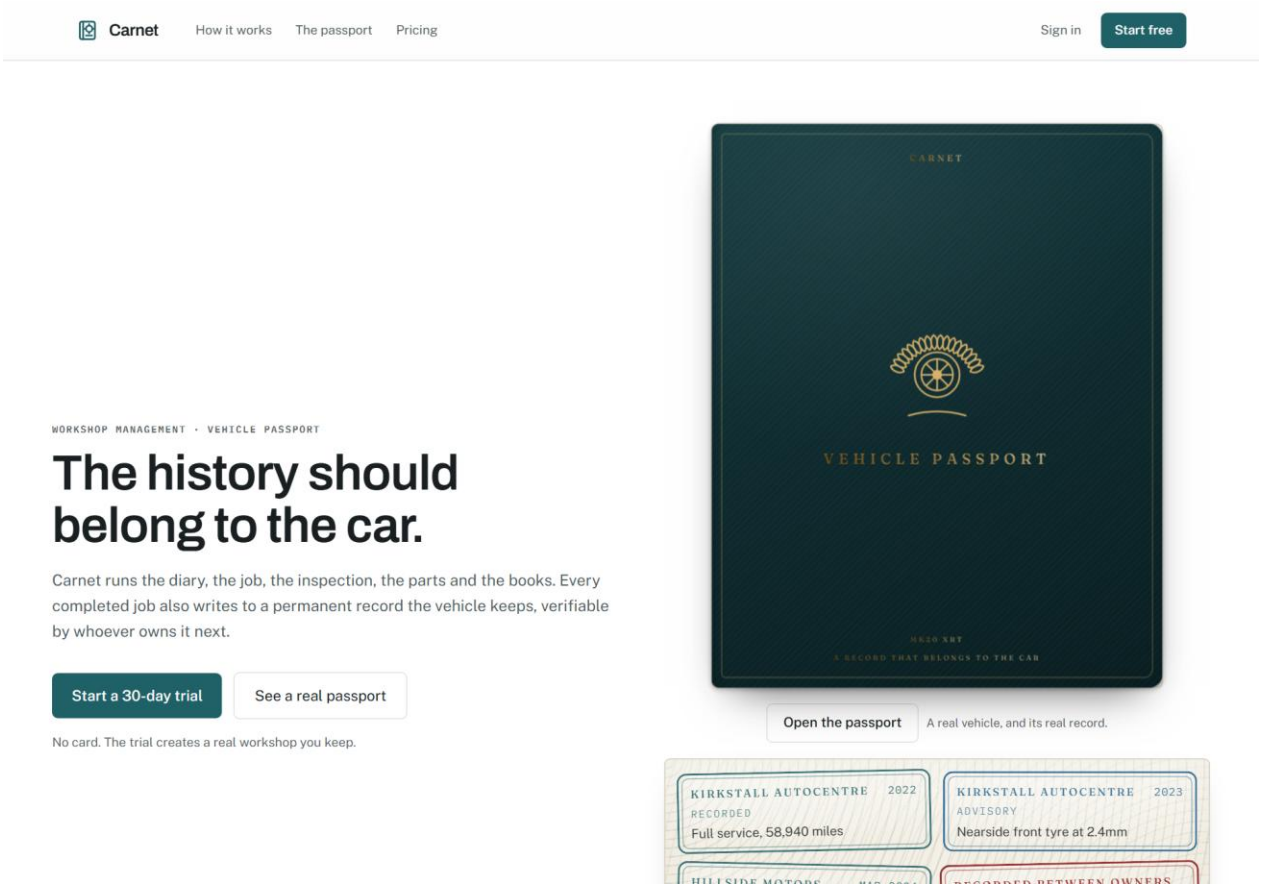
### INSPECTION CAPTURE ON A TABLET.

```
< Back        BRAKES                                        3 of 4 sections
+---------------------------------------------------------------------------+
| Front pads                                  [ Mark all in section pass]|
|  [  PASS  ] [ ADVISORY ] [ CRITICAL ] [ NOT CHECKED ]                     |
|             ^ selected                                                    |
|  Measurement  [ 1.5 ] mm                                                  |
|  Note         [ Worn to the backing plate on the outer pad.          ]    |
|               [ 🎤 dictate ]                                               |
|  Photos       [ + ]  [img]  [img]                                         |
|  Recommend    [ Replace front brake pads              ]  Est [ 420 ]      |
+---------------------------------------------------------------------------+
```

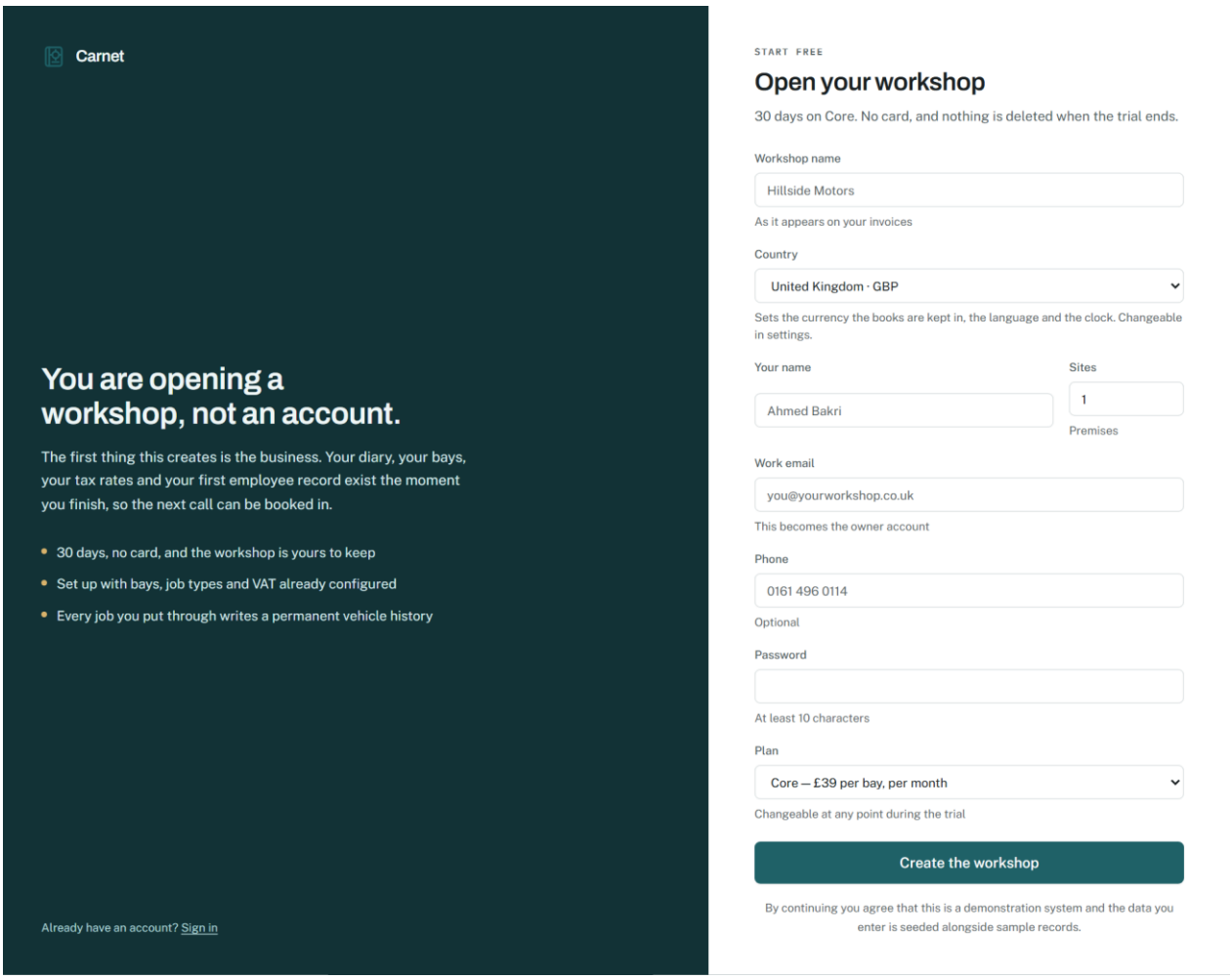
```
| Front discs                                                                 |
|  [  PASS  ] [ ADVISORY ] [ CRITICAL ] [ NOT CHECKED ]                       |
+-----------------------------------------------------------------------------+
                                                  [ Complete inspection ]
```

## 9.2 Photographs of the running application

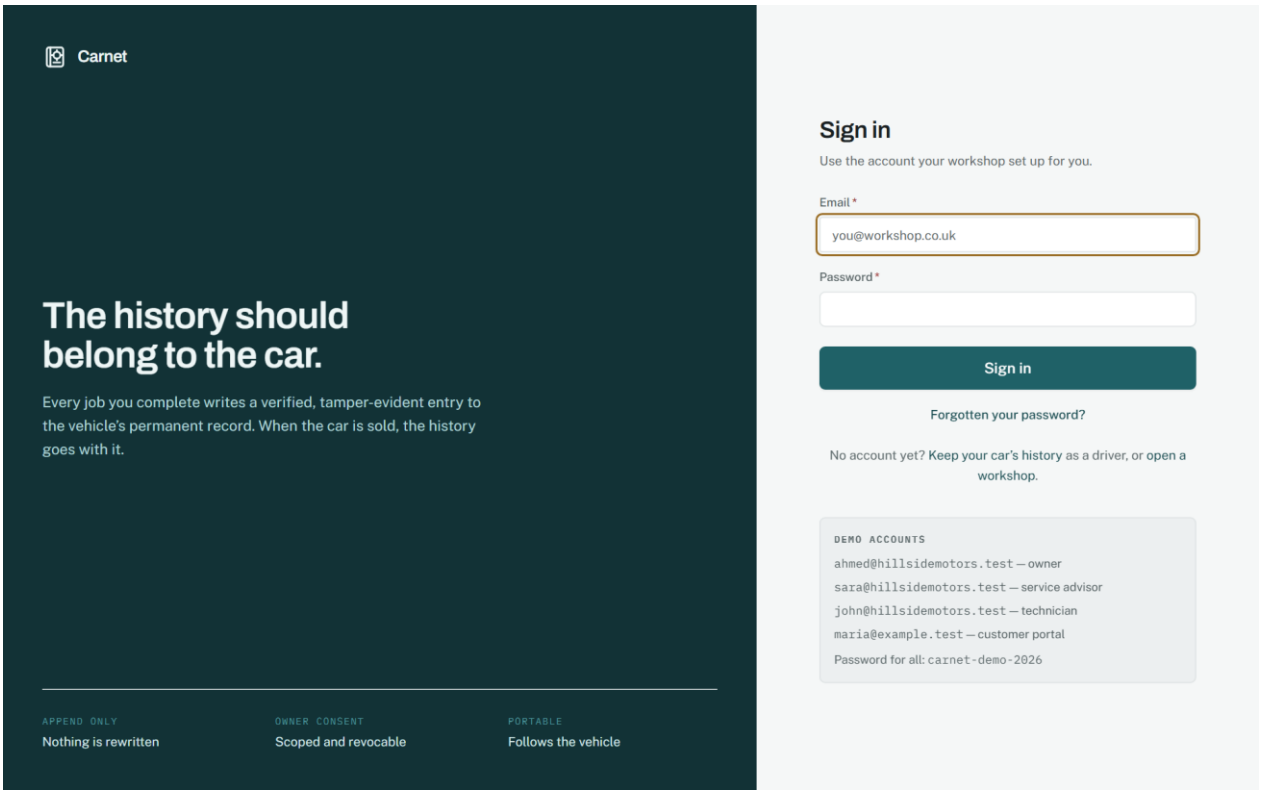
### Public




*The public front door.*

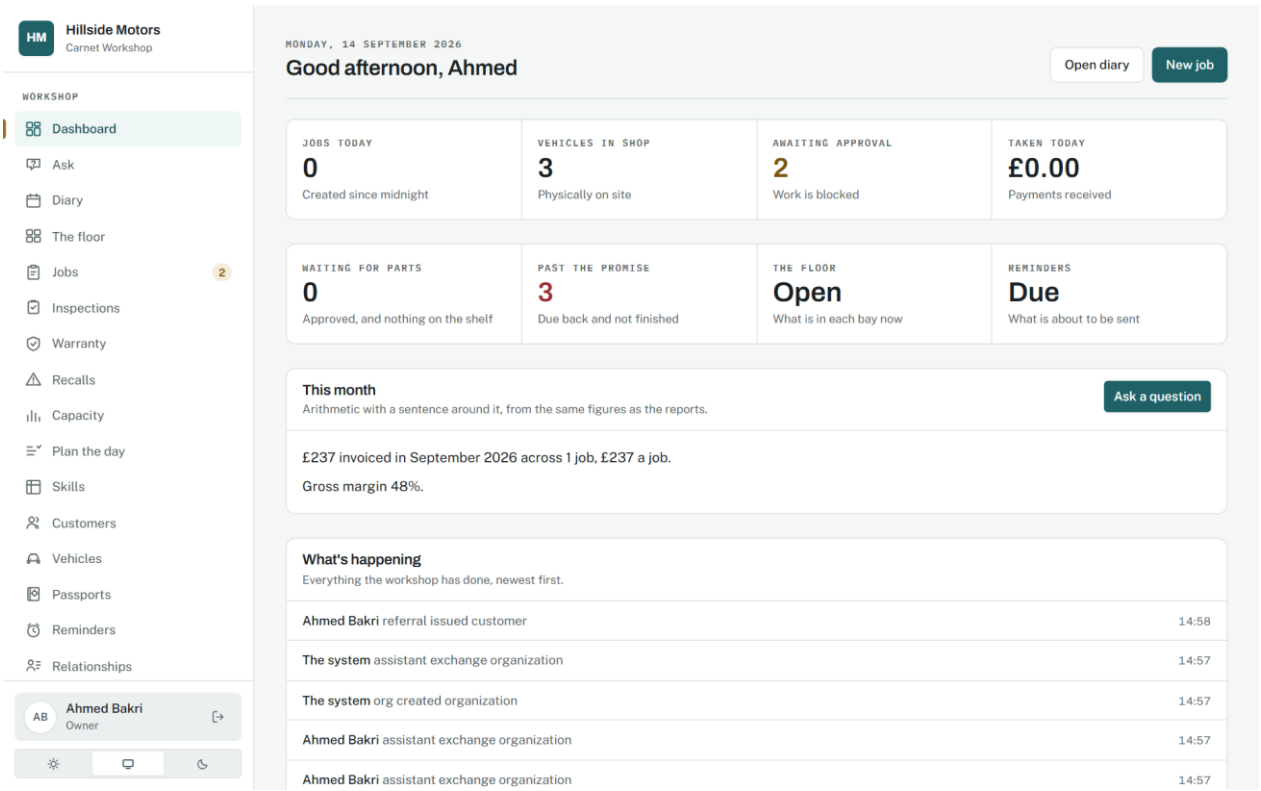



*Sign-up, with the country that sets the currency, locale, timezone and first tax rate.*

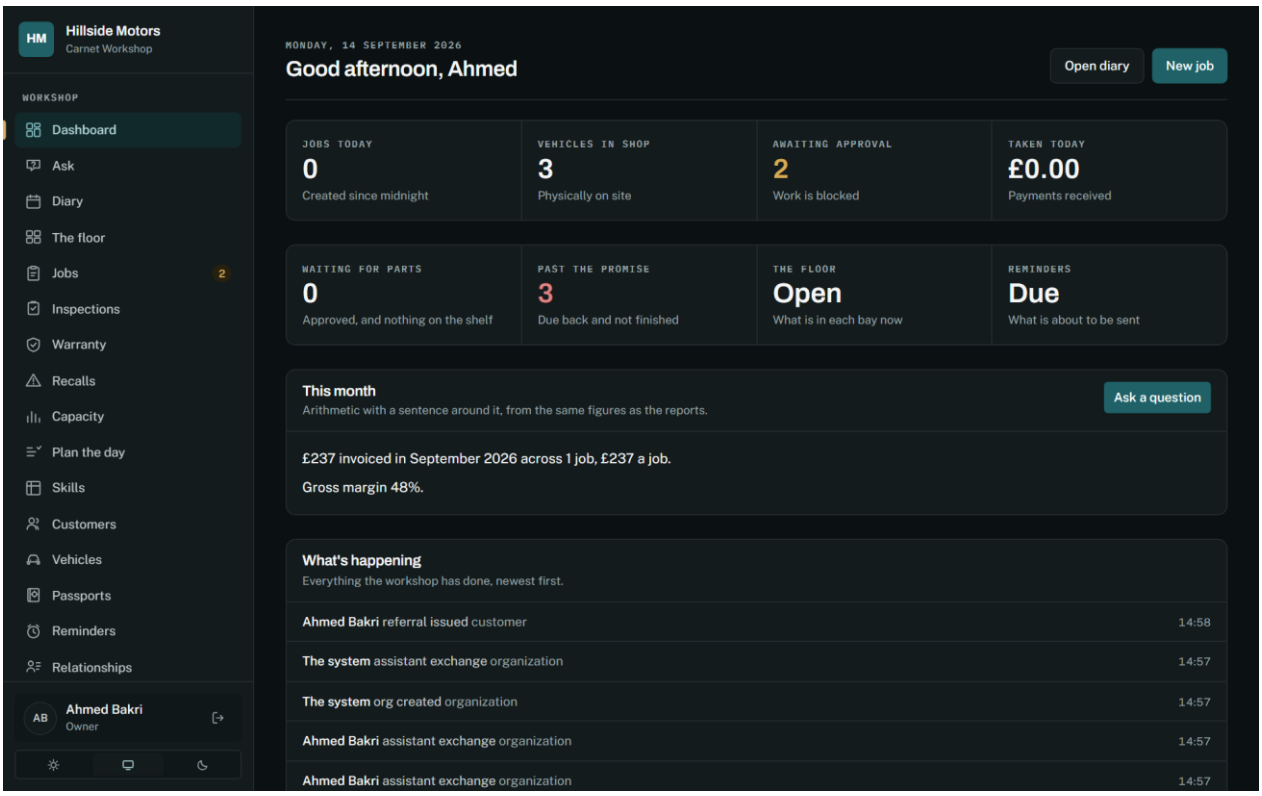



*Sign in.*

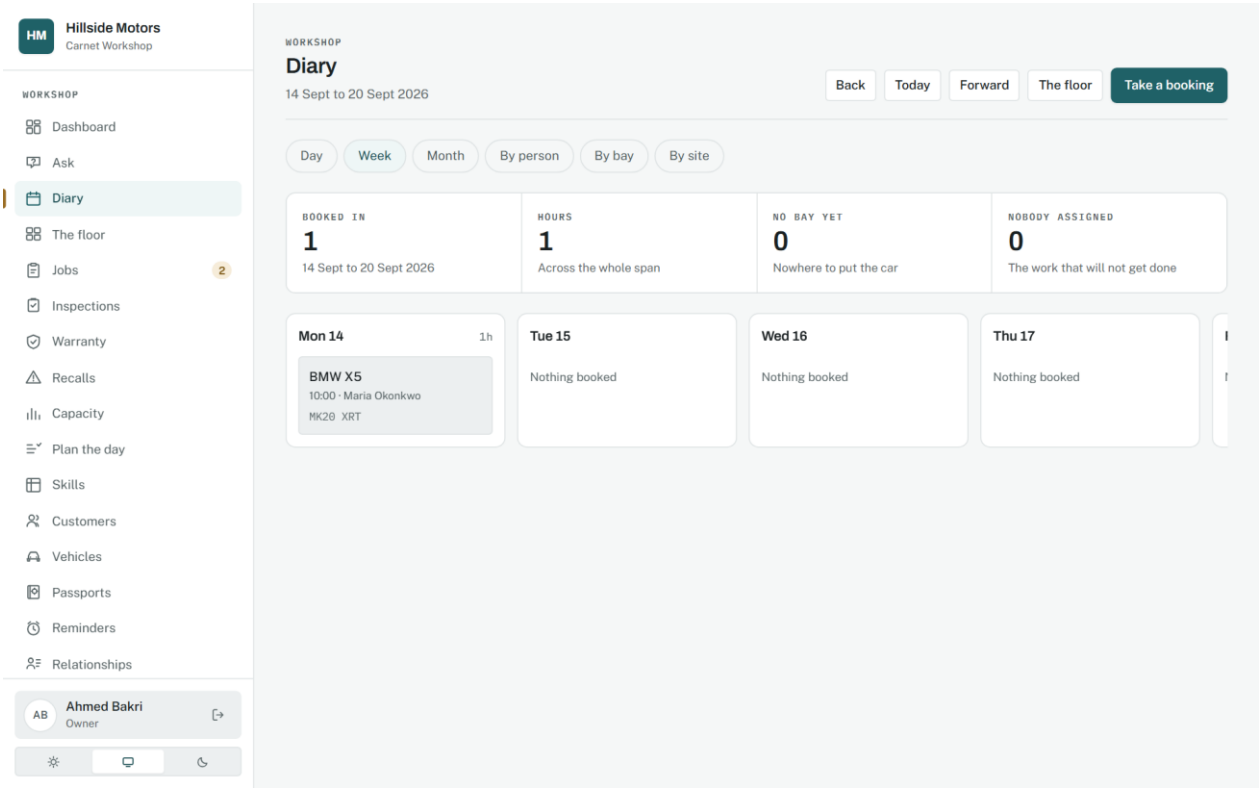
## Workshop



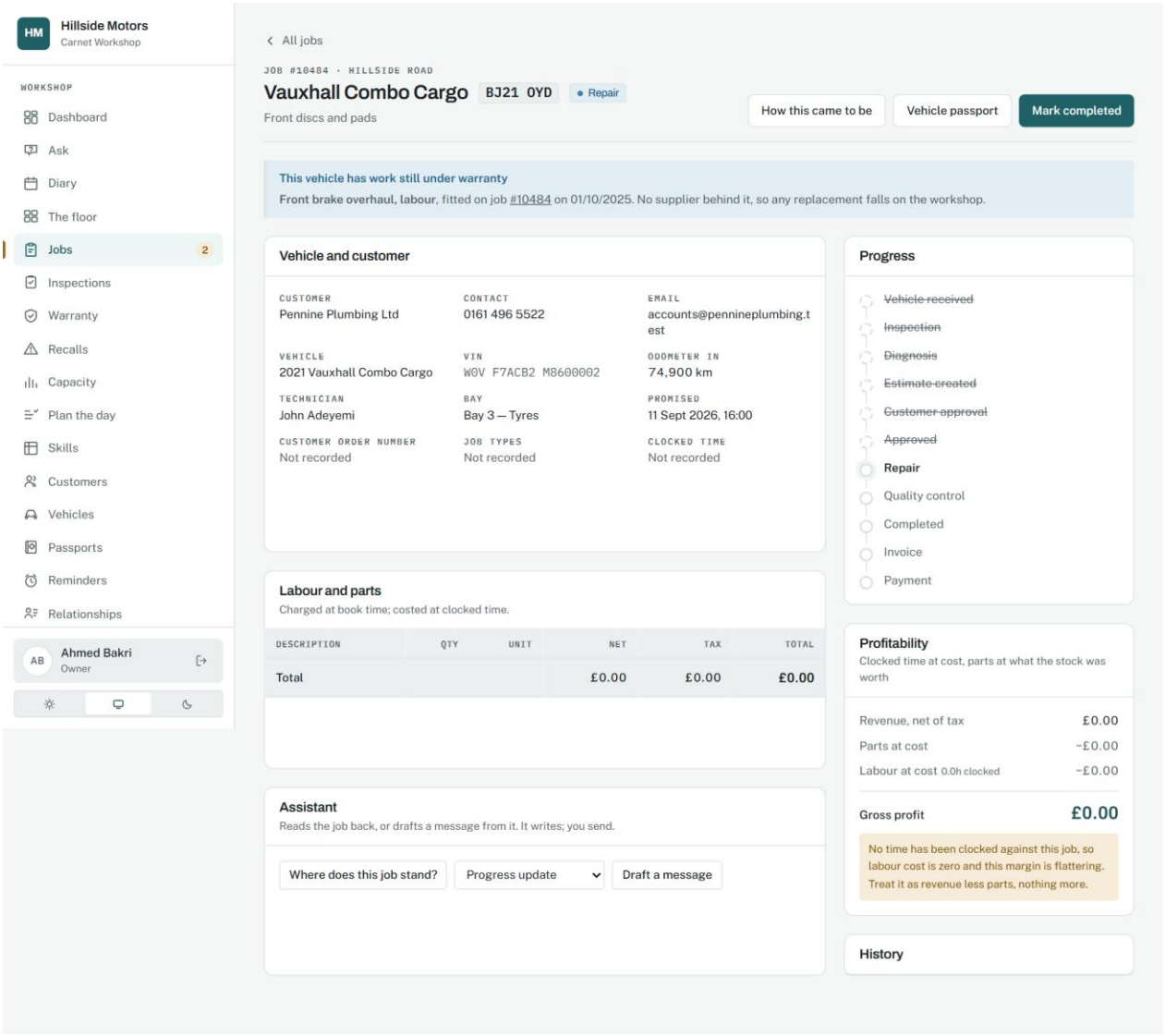

*The owner's dashboard: today's work, approvals waiting, money in and out.*




*The same, in the dark theme.*

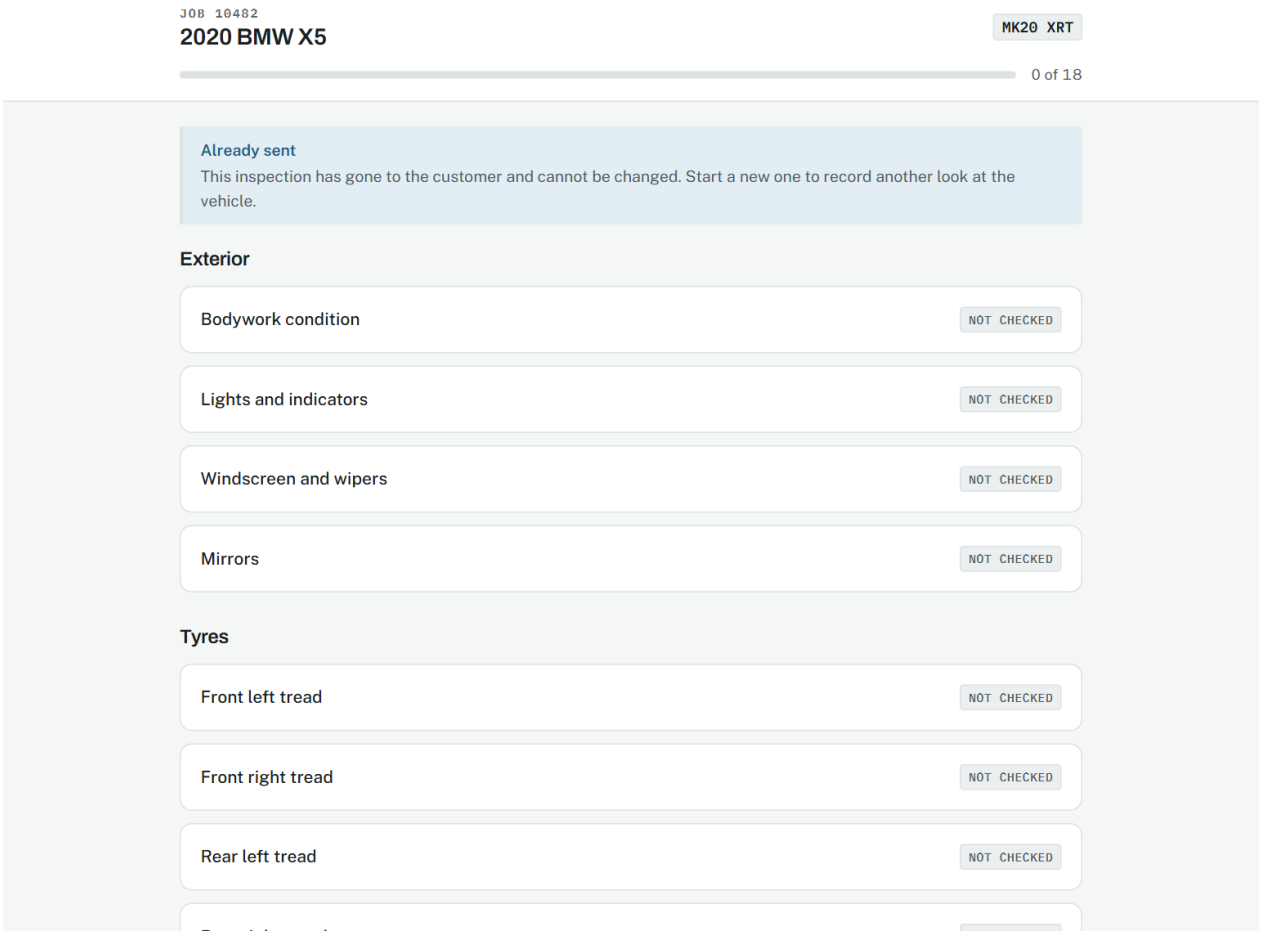



*The diary: bookings against bays with the capacity check on the drop.*




*The job: the central record, with lines, time, photographs and the timeline.*

## Shop floor




*Inspection capture, built for a tablet with gloves on.*

**Customer**

HILLSIDE MOTORS

# A quote for your 2020 BMW X5

Brake system repair · MK20 XRT

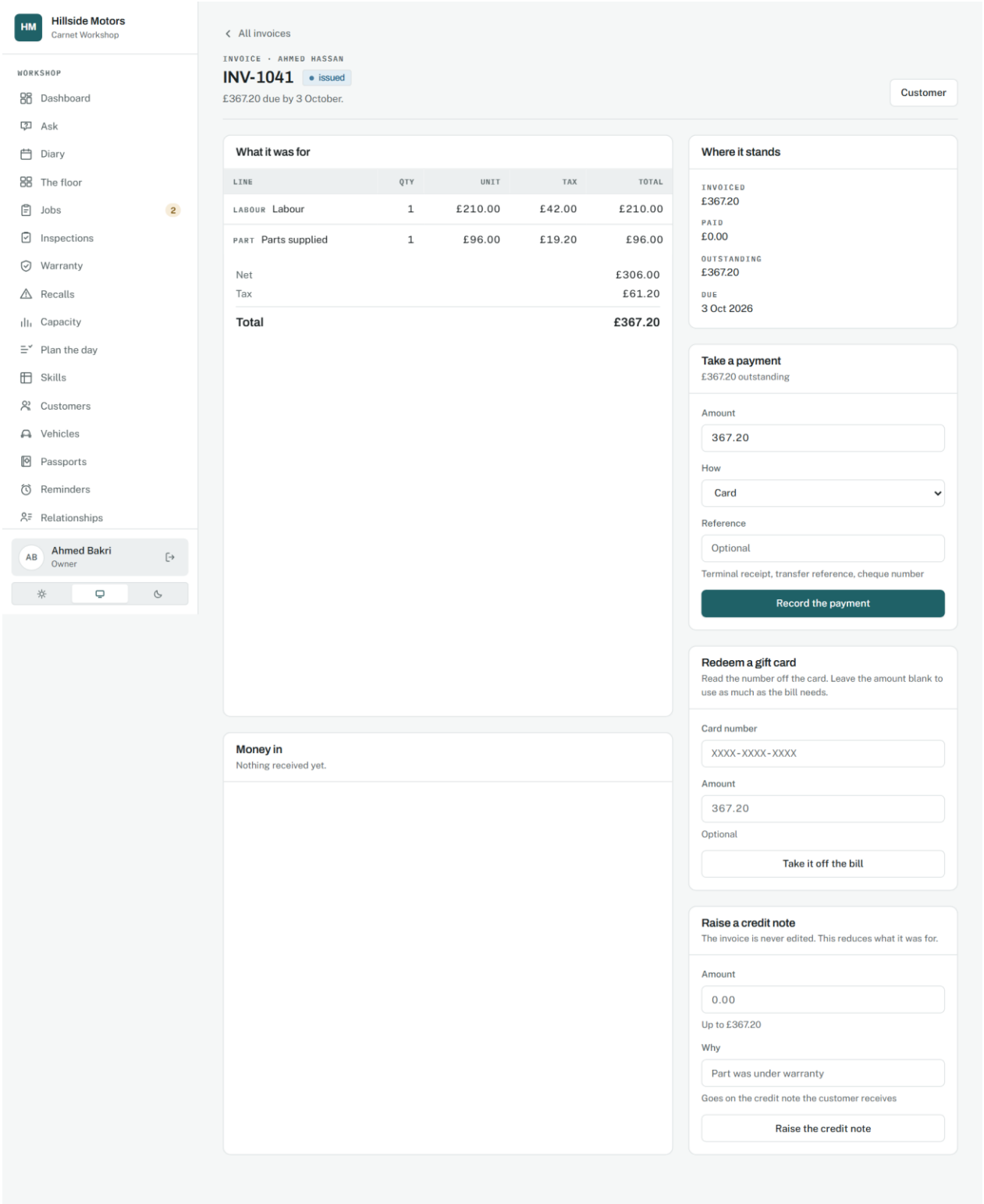
## What we recommend

Tick what you would like done. Anything left unticked will not be carried out.

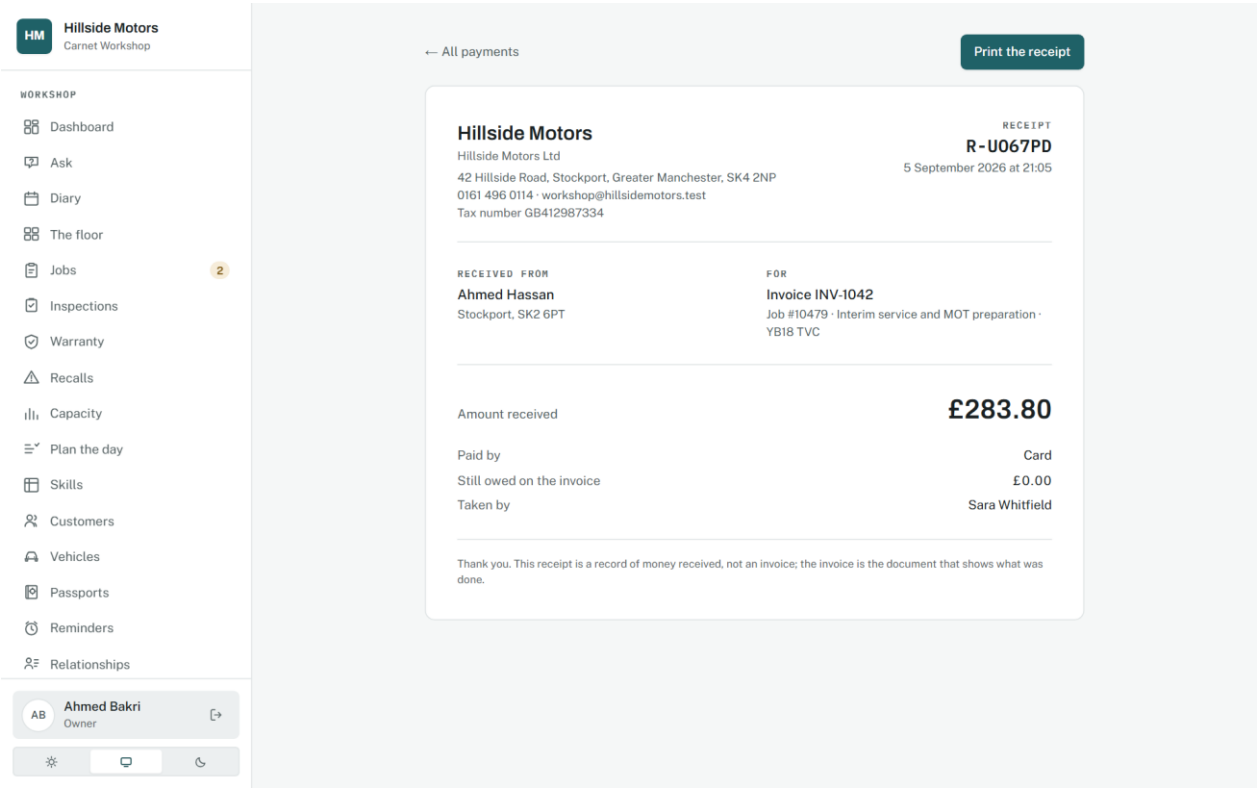
| | |
|---|---|
| ☐ Brake inspection | £95.00 |
| ☐ Front brake pad and disc replacement | £332.50 |
| ☐ BMW front brake pad set | £420.00 |
| ☐ BMW front brake discs (pair) | £410.00 |
| ☐ Front left tyre — Michelin 265/50 R19 | £165.00 |

*The estimate as the customer approves it, on their phone.*

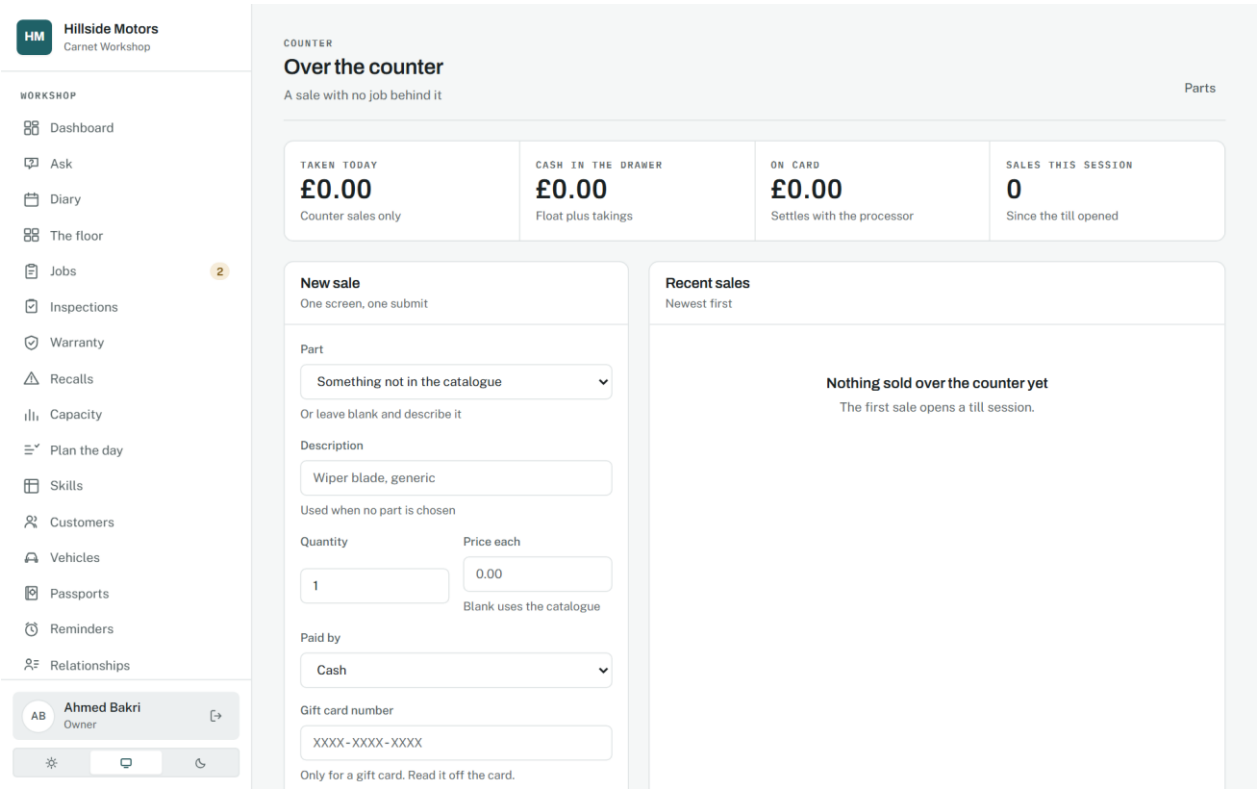
## Workshop



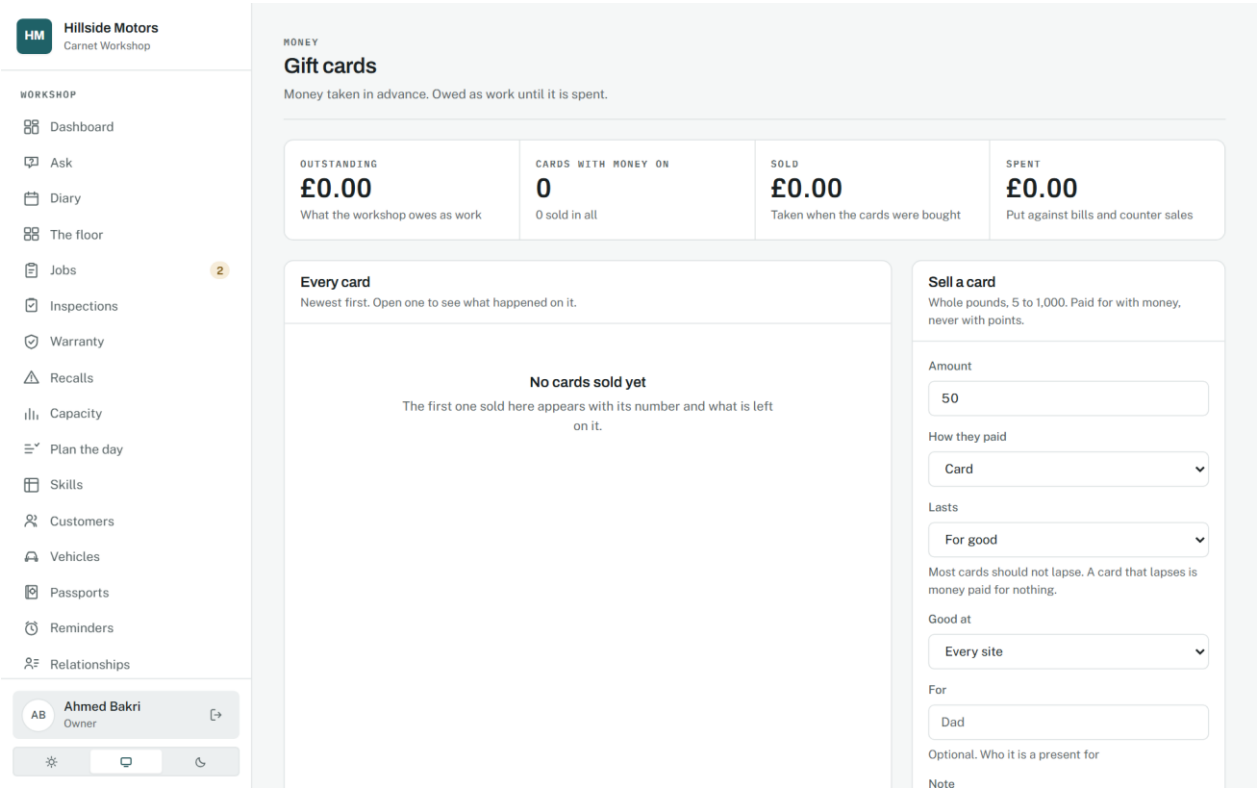

*An invoice, with what is owed and every way to take it.*



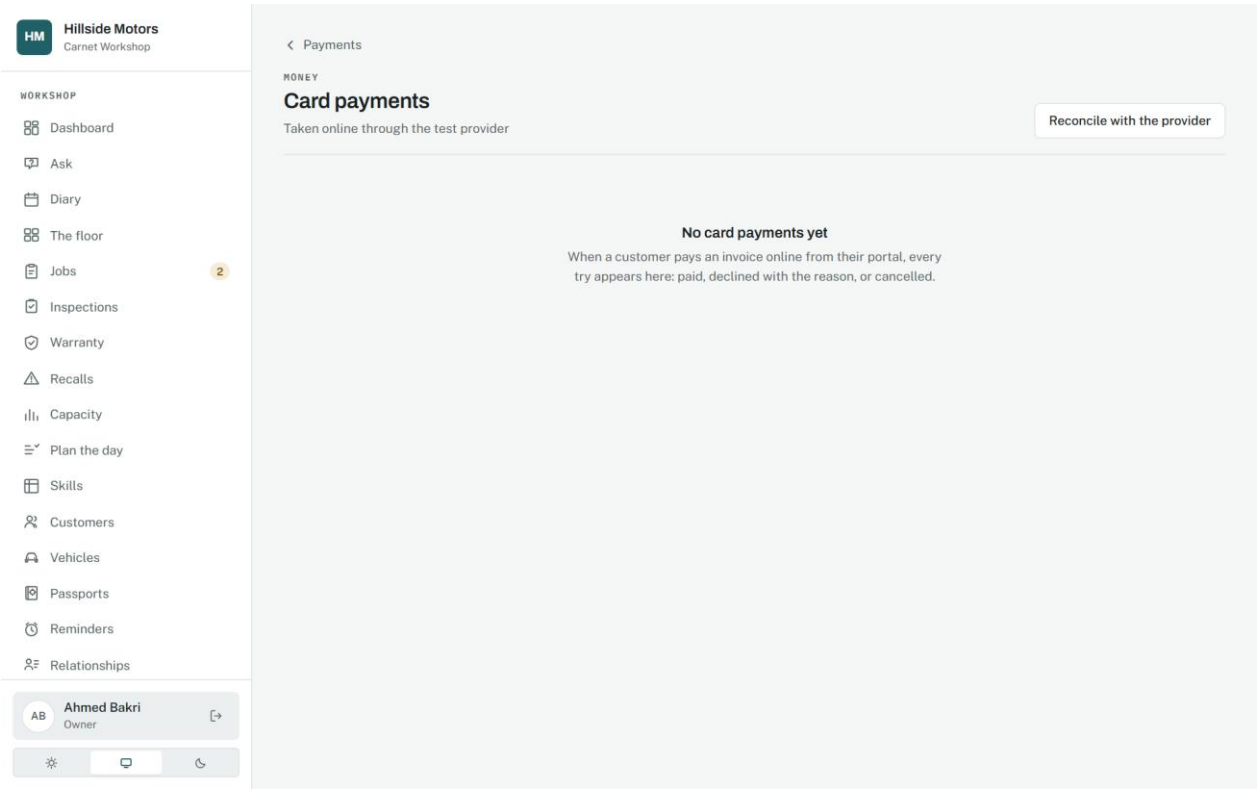

*A receipt for one payment, as it prints.*




*The counter: one sale on one screen.*




*Gift cards sold and outstanding.*




*Every card payment tried through the provider, with the provider's record against the books.*

All customers

COMPANY

# Pennine Plumbing Ltd

Source: Fleet contract

How this came to be | New job

## Contact

| MOBILE | LANDLINE | EMAIL |
|---|---|---|
| Not recorded | 0161 496 5522 | accounts@pennineplumbing.test |
| **ADDRESS** | **TAX NUMBER** | **MARKETING CONSENT** |
| Not recorded | GB338201774 | EMAIL YES · SMS NO |

## Vehicles

3 vehicles

- 2020 Vauxhall Vivaro BX70 KHT — 118,240 km · 39 history entries — Passport
- 2021 Vauxhall Combo Cargo BJ21 OYD — 74,900 km · 0 history entries — Passport
- 2019 Ford Transit Custom BV19 FLM — Not recorded · 0 history entries — Passport

## Recent jobs

| | | | | |
|---|---|---|---|---|
| #10486 | Rear coil springs | Vauxhall Vivaro | 11/09/2026 | PAYMENT |
| #10485 | Battery and alternator belt | Ford Transit Custom | 11/09/2026 | PAYMENT |
| #10484 | Front discs and pads | Vauxhall Combo Cargo | 11/09/2026 | REPAIR |
| #10483 | Clutch judder under load | Vauxhall Vivaro | 11/09/2026 | CUSTOMER APPROVAL |

## Preferences

Preselected on a new job; the diary has the last word

Asks for: Nobody in particular

Save preference

## Account

Outstanding £1,284.00

Invoices 1

## Money held

A deposit before the work, or credit kept from a refund

Held £0.00

Deposit: 200.00 — How they paid: Card

For which job: Not a particular job

Optional

Reference

Optional

Take the deposit

## Points

1 point a pound on every bill paid in full

Balance 0

Nothing earned yet. The first bill they pay in full starts it.

Points — Why: Sorry about the wait on Tu

Add points

## Referrals

Both sides get 500 points once the person brought in has had £50 of work done

THEIR CODE TO HAND OUT

**6NNK7D**

Nobody brought in yet.

Arrived with a code: XXXXXX

Somebody else's code, said at the counter

Use the code

## Recent payments

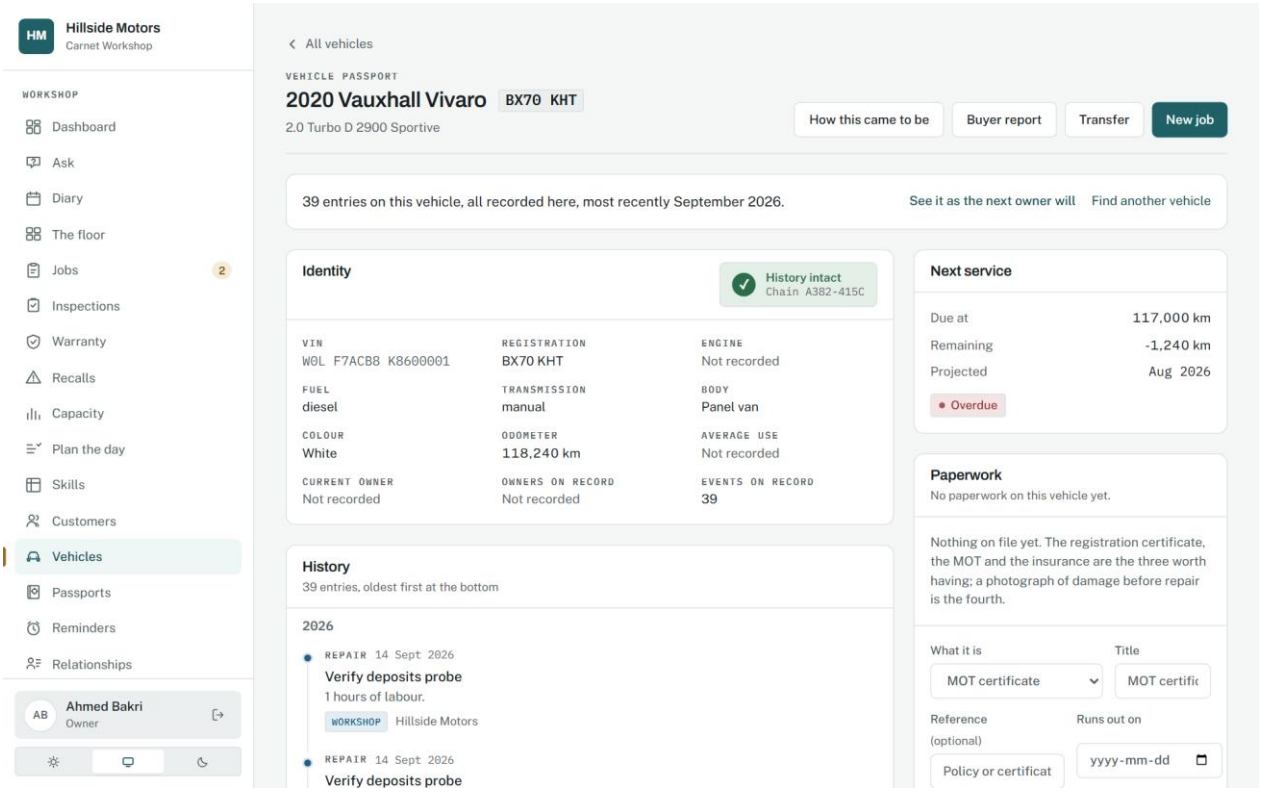
Nothing recorded.

## The whole story

Every visit, quote, decision and payment, newest first.

| | | |
|---|---|---|
| 13 Sept 26 | Booked in for Cambelt and water pump | |
| 11 Sept 26 | Vauxhall Combo Cargo came in for Front discs and pads Job 10484 | |
| 11 Sept 26 | Vauxhall Vivaro came in for Clutch judder under load Job 10483 | |
| 11 Sept 26 | Booked in for Diagnostics — intermittent EML | |
| 24 May 26 | Invoiced INV-1036 | £1,284.00 |

*A customer, with their vehicles, bills, money held and points.*




*A vehicle as the workshop sees it.*

PASSPORT

# As the next owner will see it

The document a buyer receives through a share link, word for word. No owner is named on it.




## PARTICULARS OF THE VEHICLE 01

Vauxhall
Vivaro

BX70 KHT

HISTORY INTACT
A382-415C

### 2020 Vauxhall Vivaro

| | |
|---|---|
| TYPE: P | CODE: CNT |
| PASSPORT NUMBER: A382-415C | REGISTRATION: BX70 KHT |
| VEHICLE IDENTIFICATION NUMBER: W0LF7ACB8K8600001 | |
| MAKE: Vauxhall | MODEL: Vivaro |
| YEAR: 2020 | FIRST REGISTERED: 17 Sept 2021 |
| ENTRIES ON RECORD: 39 | WORKSHOPS CONTRIBUTING: 1 |
| OWNERS ON RECORD: 0 | LATEST ODOMETER: 118,240 km |
| MILEAGE ANOMALIES: None | OPEN RECALLS: None |

This record is append-only. Each entry is linked to the one before it, so an entry cannot be altered or removed without breaking the chain. The seal recomputes it.

```
P<CNTVAUXHALL<<VIVARO<<2020<<<<<<<<<<<<<<<
W0LF7ACB8K86000019BX70<KHT<92109170000039<2<
```

## OBSERVATIONS 02

WHAT THIS RECORD AMOUNTS TO

**Intact, but thin. Read what it does not cover as carefully as what it does.**

WHAT IT DOES NOT COVER

The record starts in September 2026, and the vehicle was first registered in September 2021. About 4 years before that is not covered here at all.

Nothing here was inspected by a workshop, so the condition of the vehicle is not part of this record.

This covers what participating workshops recorded. Work done elsewhere, or by the owner, does not appear.

WHO HAS OWNED IT

No ownership on record.

No owner is named on these pages, past or present. That is theirs, not the vehicle's.

## MILEAGE 03

**The mileage record**

Checked across every workshop that has recorded a reading. 0 on file.

Too few readings to say much. 0 readings on record.

This checks the readings against each other. It cannot see mileage covered while nobody was writing anything down, and it is not a warranty.

## ENTRIES · 2026 04

Newest first. Each stamp names who pressed it and how well evidenced the entry is.

HILLSIDE MOTORS — 14 SEPT 2026
REPAIR
Verify deposits probe
1 hours of labour.

HILLSIDE MOTORS — 14 SEPT 2026
REPAIR
Verify deposits probe
3 hours of labour.

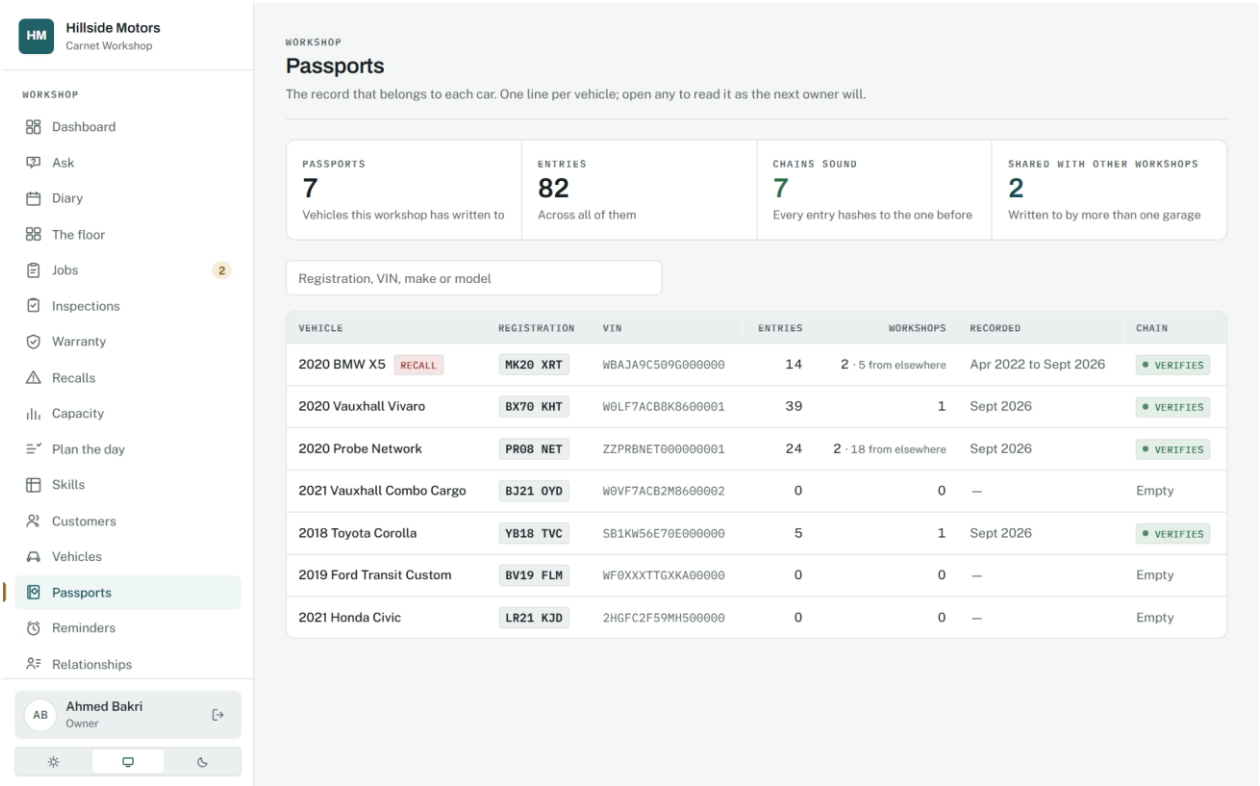
HILLSIDE MOTORS — 14 SEPT 2026
REPAIR
Verify referrals probe
2 hours of labour.

HILLSIDE MOTORS — 14 SEPT 2026
REPAIR
Verify referrals probe
0.5 hours of labour.

HILLSIDE MOTORS — 14 SEPT 2026
REPAIR

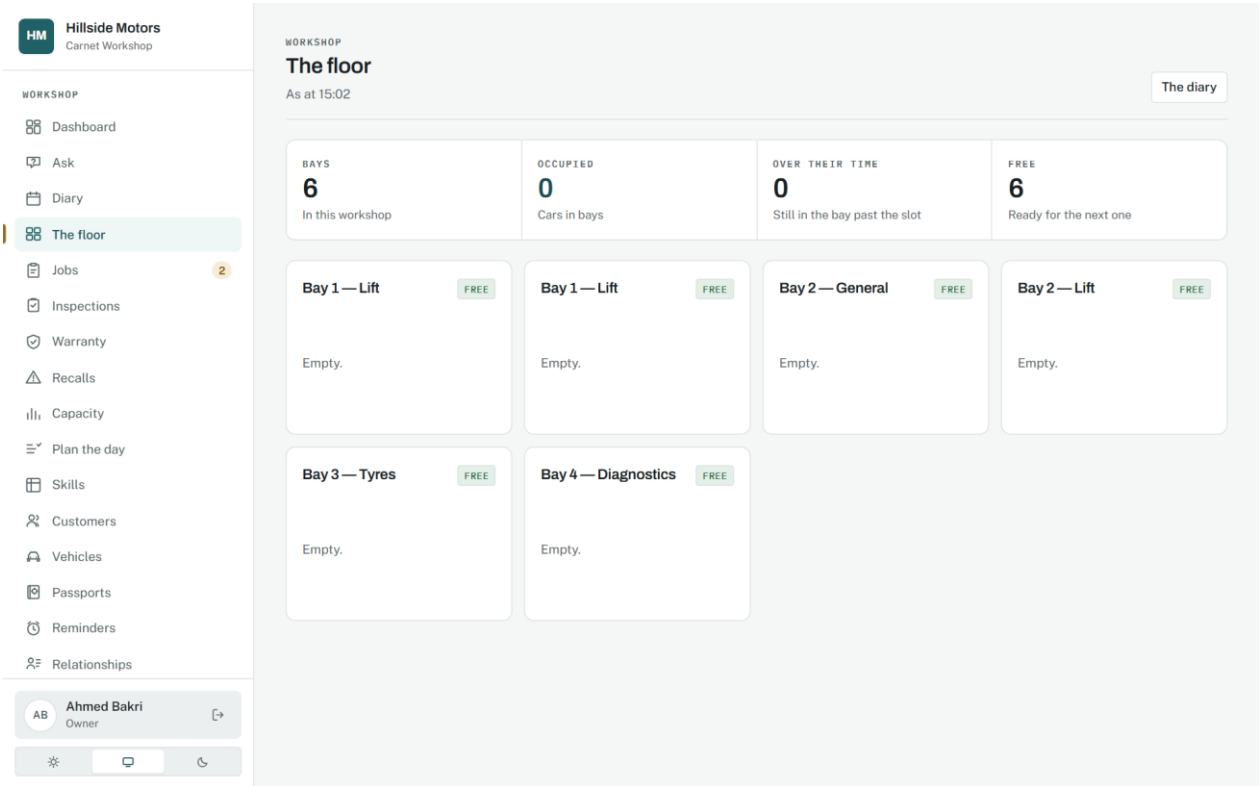
HILLSIDE MOTORS — 14 SEPT 2026
REPAIR

*The vehicle passport: the hash-chained history, verified on the page.*

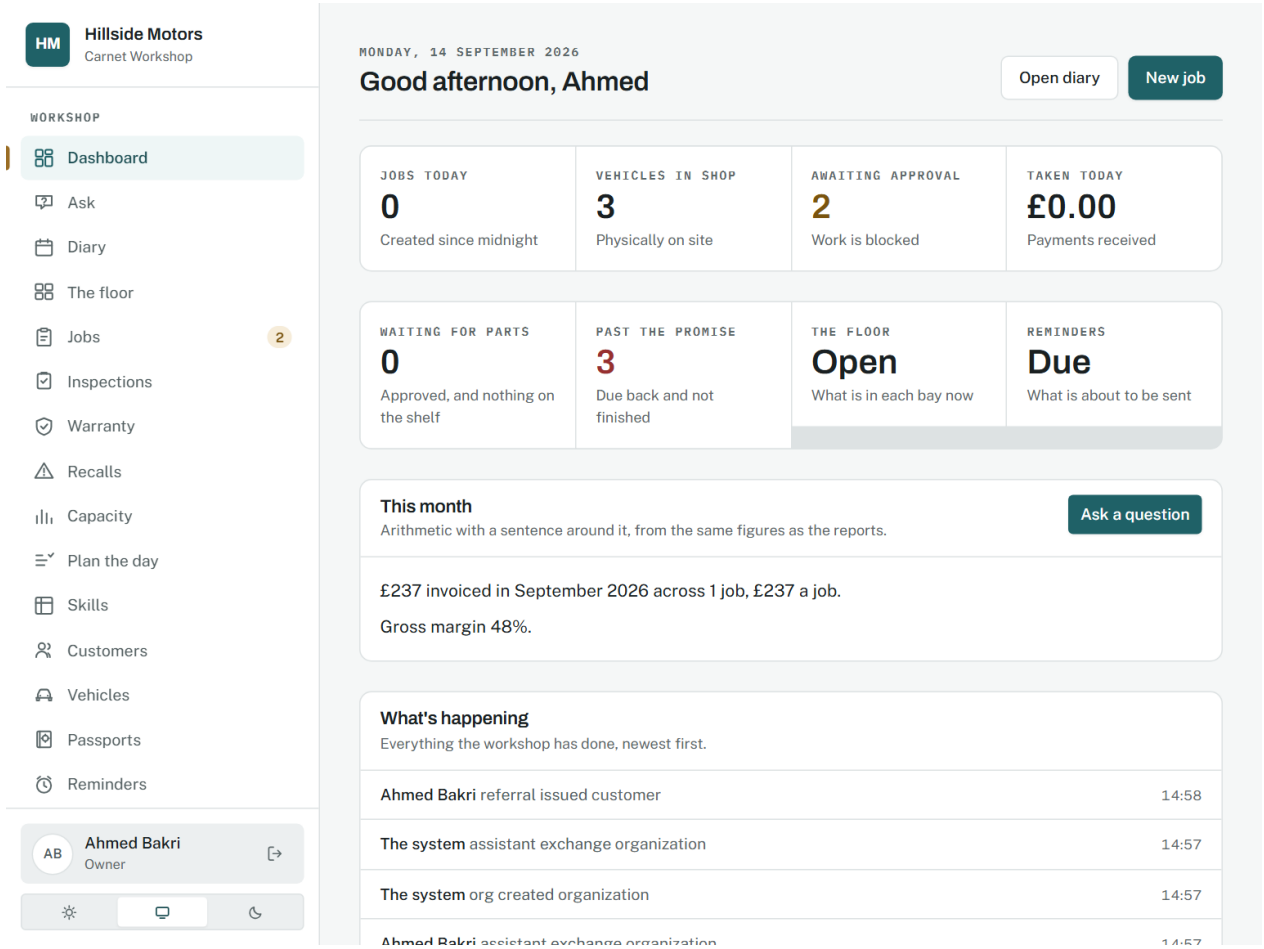



*Every passport the workshop has touched, with its chain verdict.*

## Shop floor

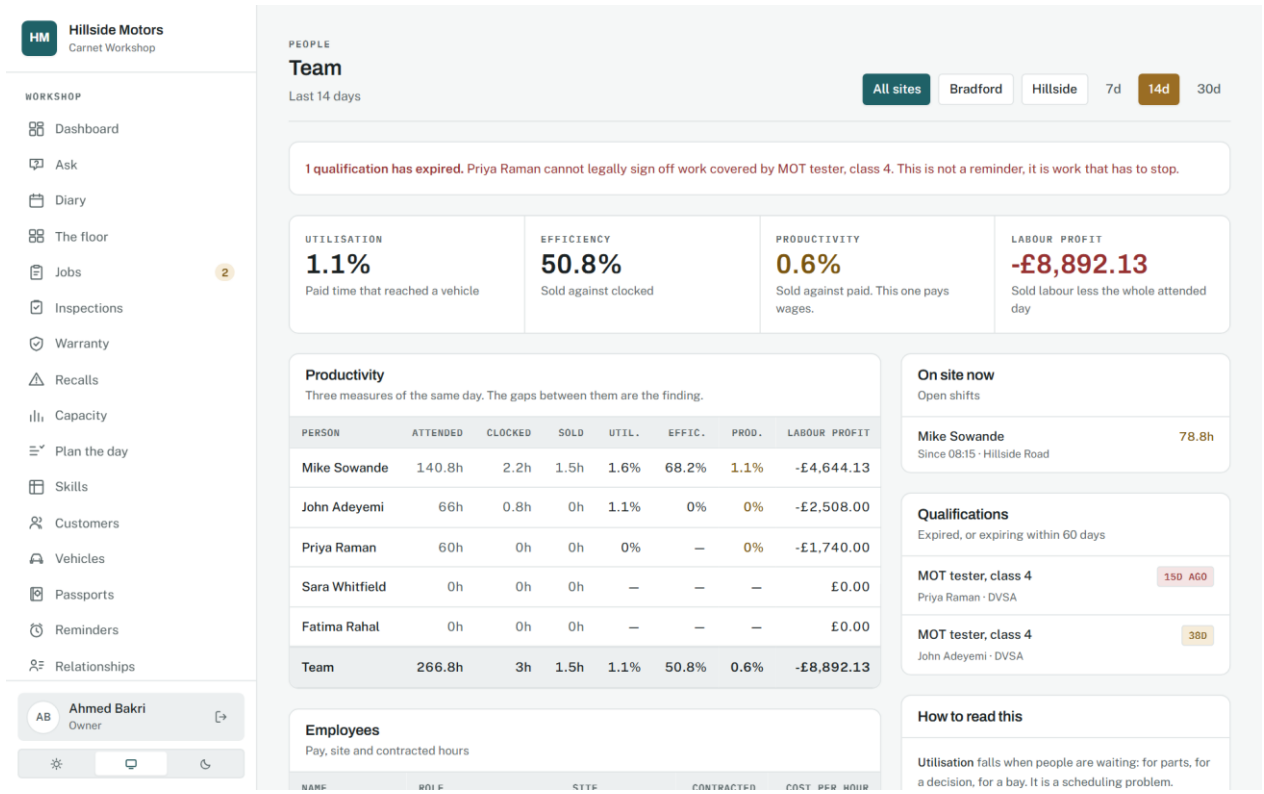



*The shop floor board.*

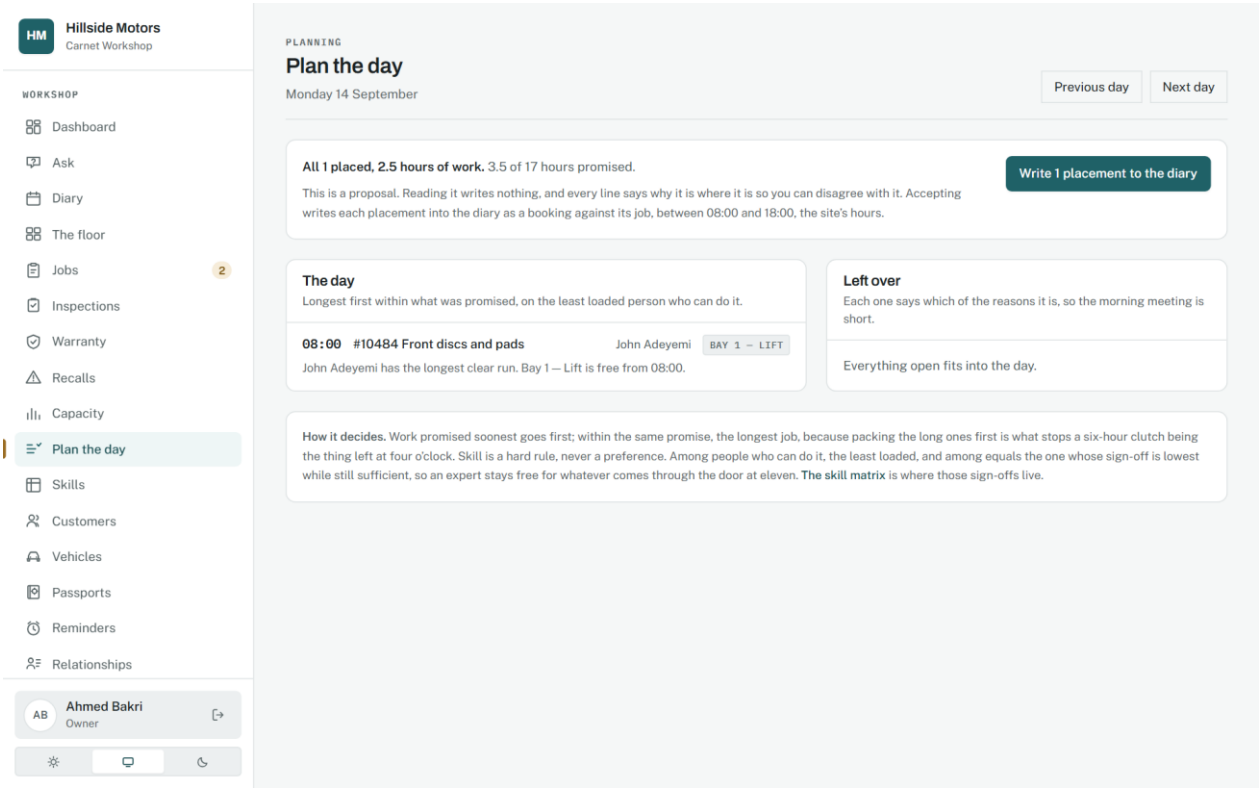



*The technician's own surface.*

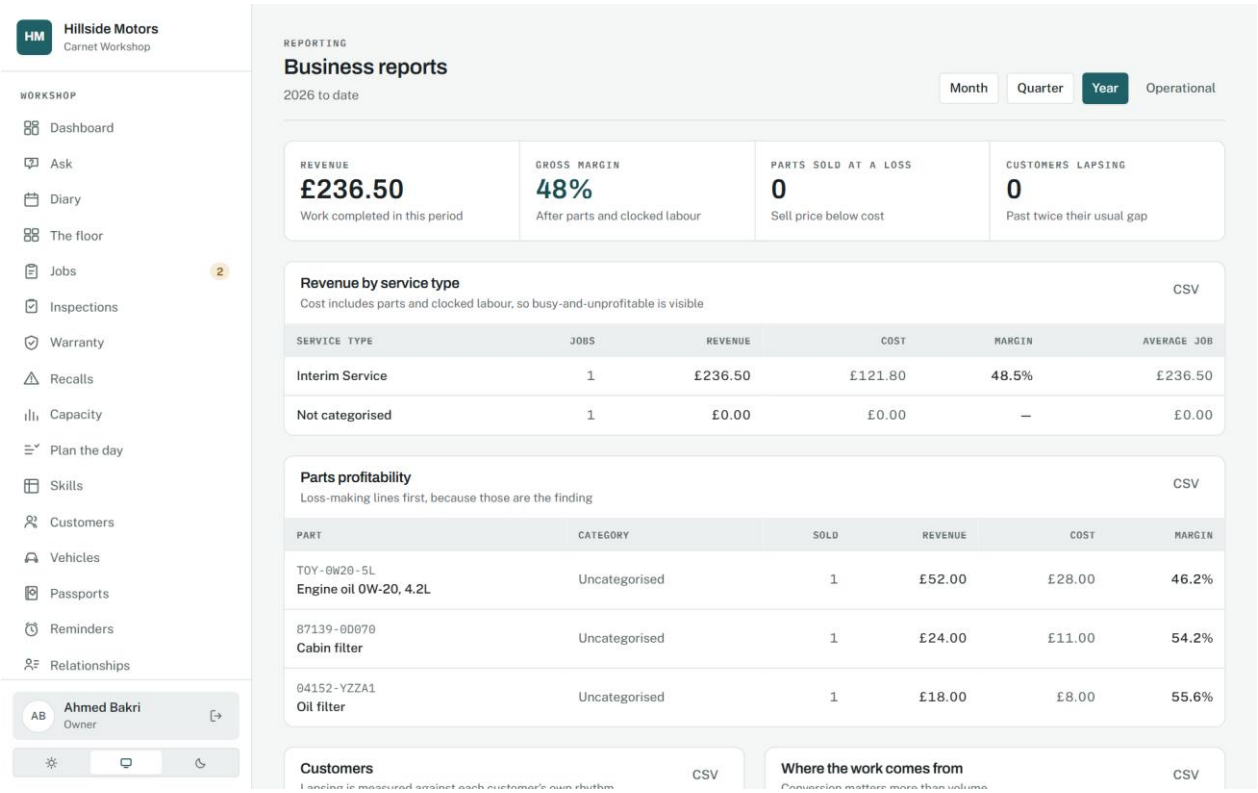
## Workshop



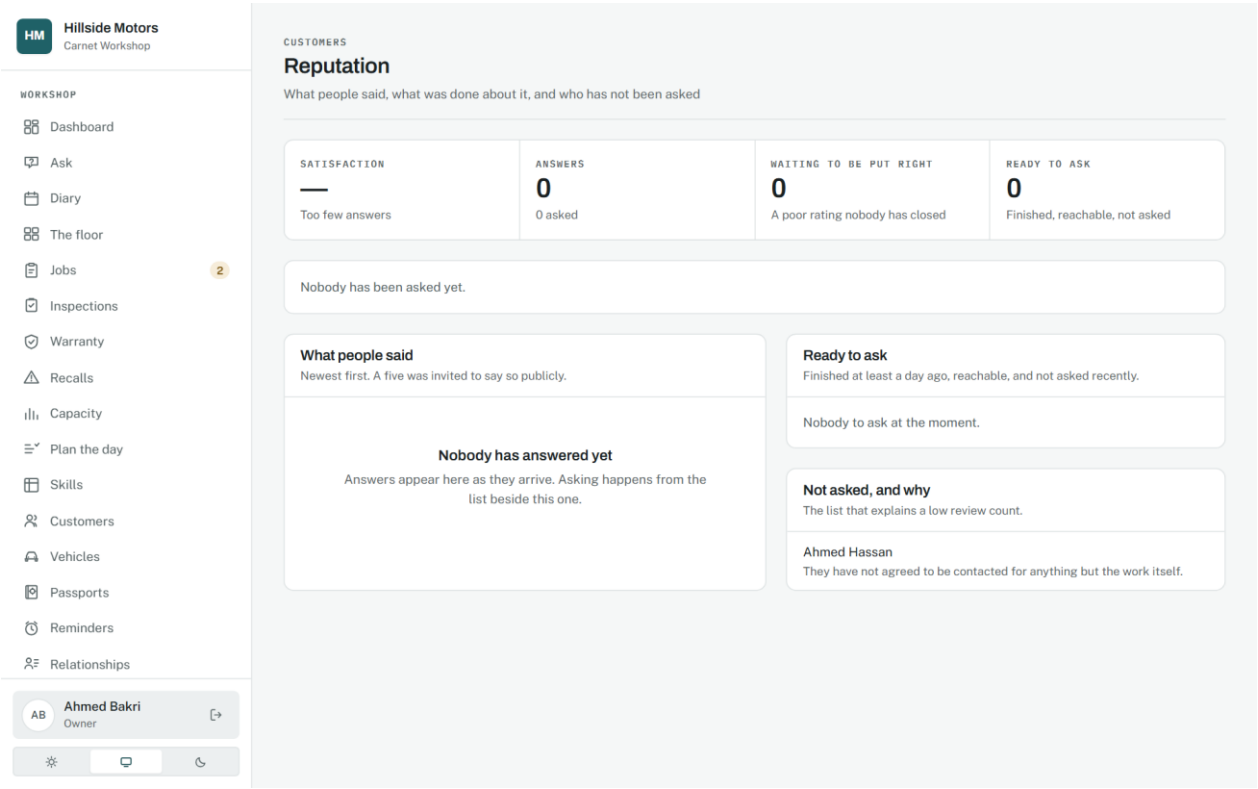

*People, skills and the week.*



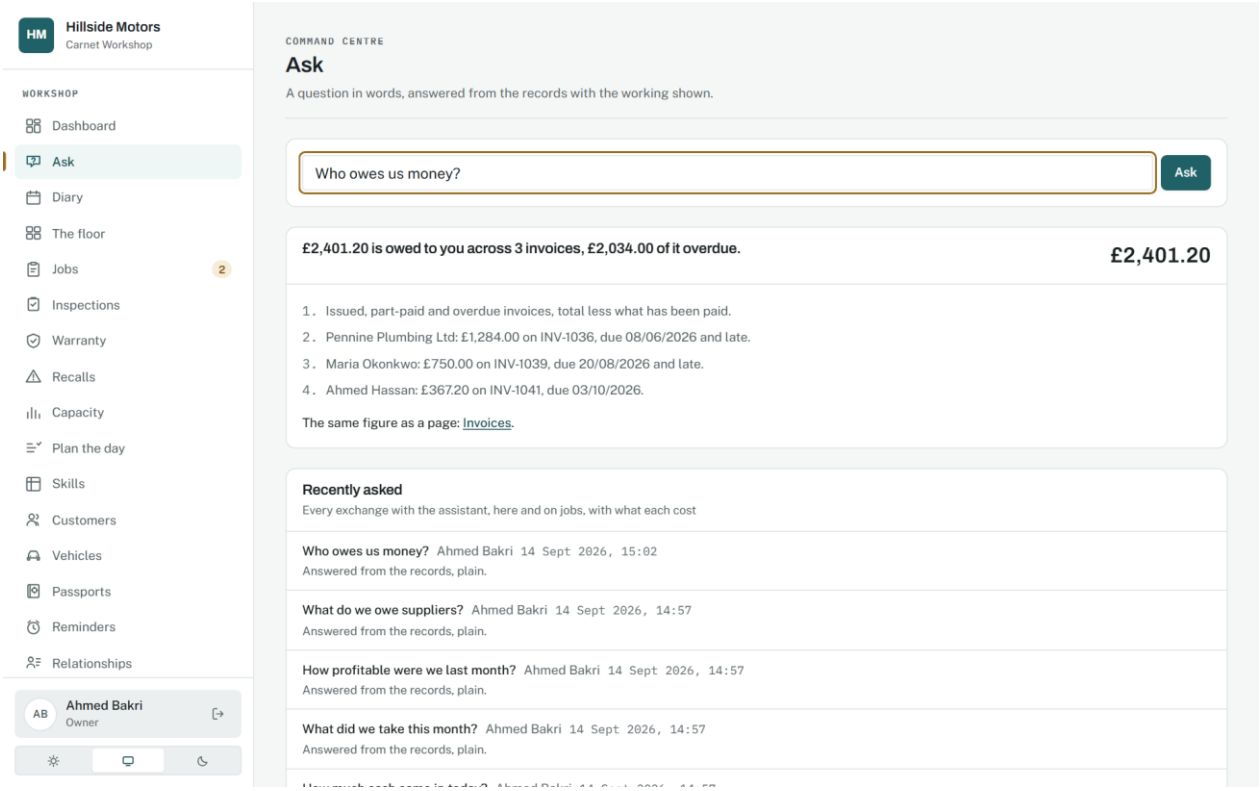

*The planner: capacity against demand.*



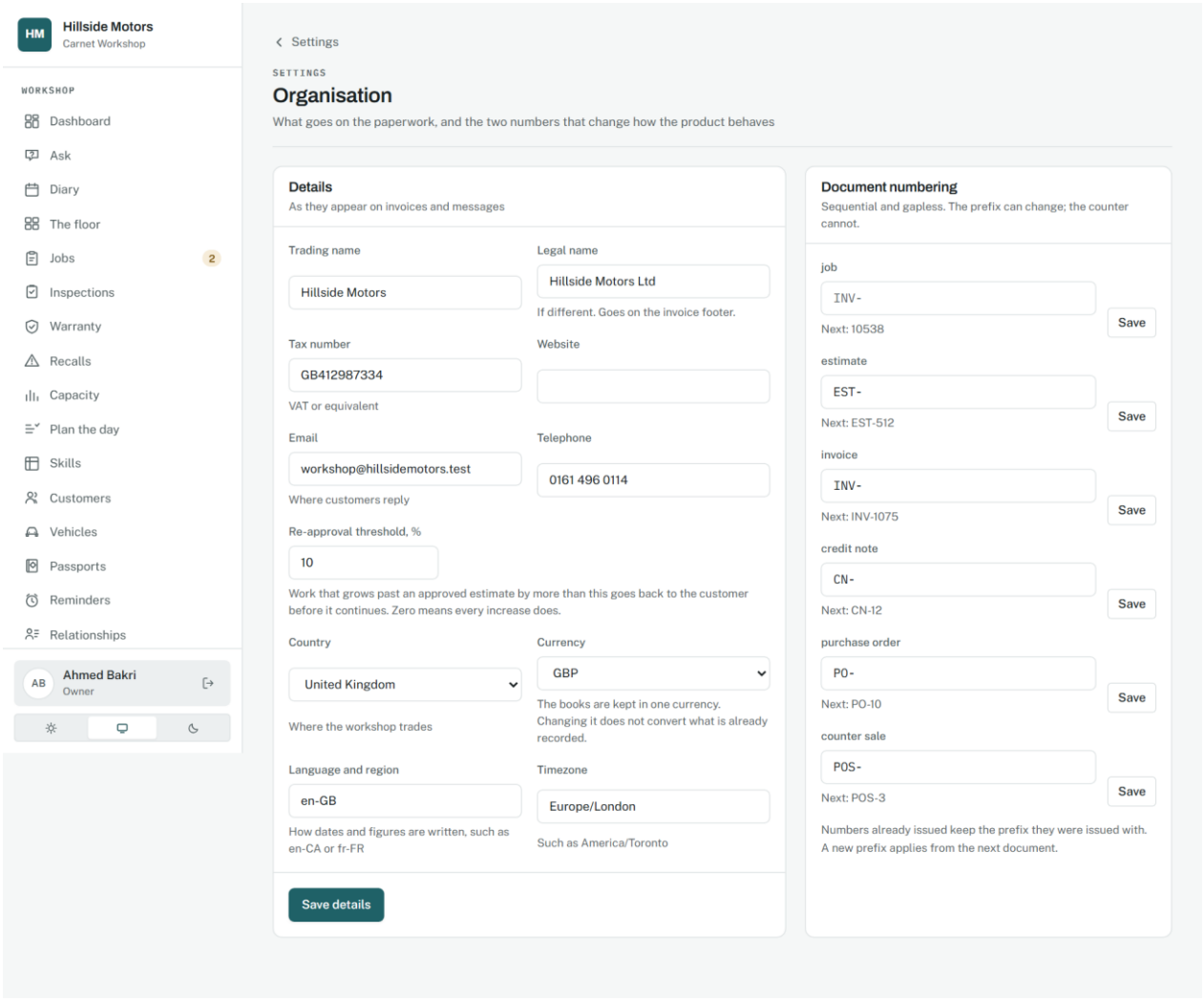

*Business reporting.*



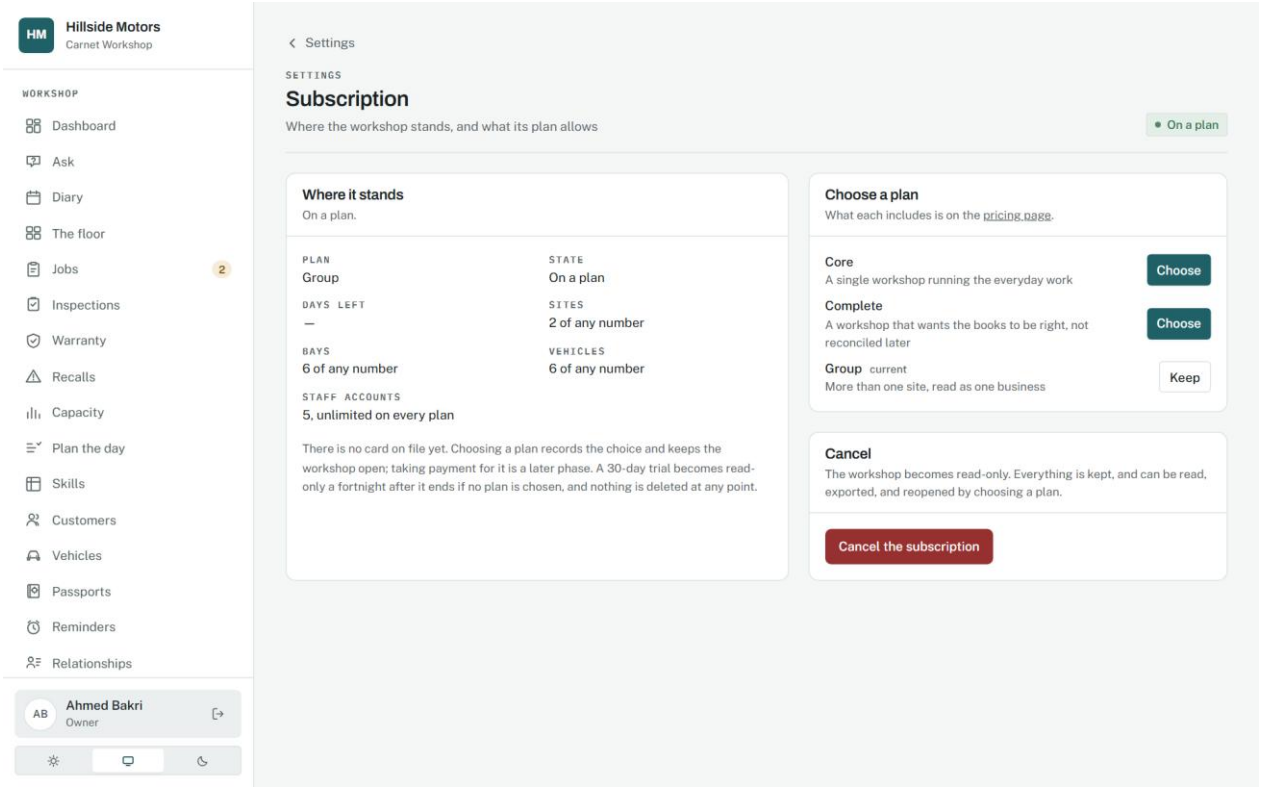

*Reviews and reputation.*




*Ask: a question in words, answered from the workshop's own data.*




*Organisation settings, where the region is changed.*



*The subscription page: where the workshop stands and what its plan allows.*

**Customer**

Carnet

Sign out

# Hello, Maria

1 estimate needs your decision.

**BMW X5**

Hillside Motors · Brake system repair

Needs your approval

ESTIMATED TOTAL

**£1,509.00**

Review and decide

## My vehicles

Add another car

Book a visit

### 2020 BMW X5 MK20 XRT

82,430 km · 14 entries on record

Customer approval

This vehicle has an outstanding safety recall. Contact your workshop.

View history

Share with a buyer

*The customer's portal.*

Carnet

Sign out

My vehicles Passports Updates Invoices

Book a visit

# Invoices

1 still to pay.

## Outstanding £750.00

**INV-1039** UNPAID

Hillside Motors · issued 21 July 2026

Counter sale

| | |
|---|---|
| Invoiced | £750.00 |
| **Still to pay** | **£750.00** |

Pay £750.00 now

Due 20 August 2026. Paid by card through a secure checkout, or pay the workshop directly.

*The customer's invoices, with Pay now on what is owed.*

Carnet

Sign out

My vehicles Passports Updates Invoices

Book a visit

← Back to the vehicle

Show this to somebody




PARTICULARS OF THE VEHICLE 01

*A passport as its owner sees it.*

Carnet

Sign out

My vehicles Passports Updates Invoices

Book a visit

# Your passports

2 cars, 14 entries, every record sound.

## 2020 BMW X5 MK20 XRT

14 entries from 2 garages, since April 2022.

- Record sound

This vehicle has an outstanding safety recall. Contact your workshop.

Open the passport

Show it to a buyer

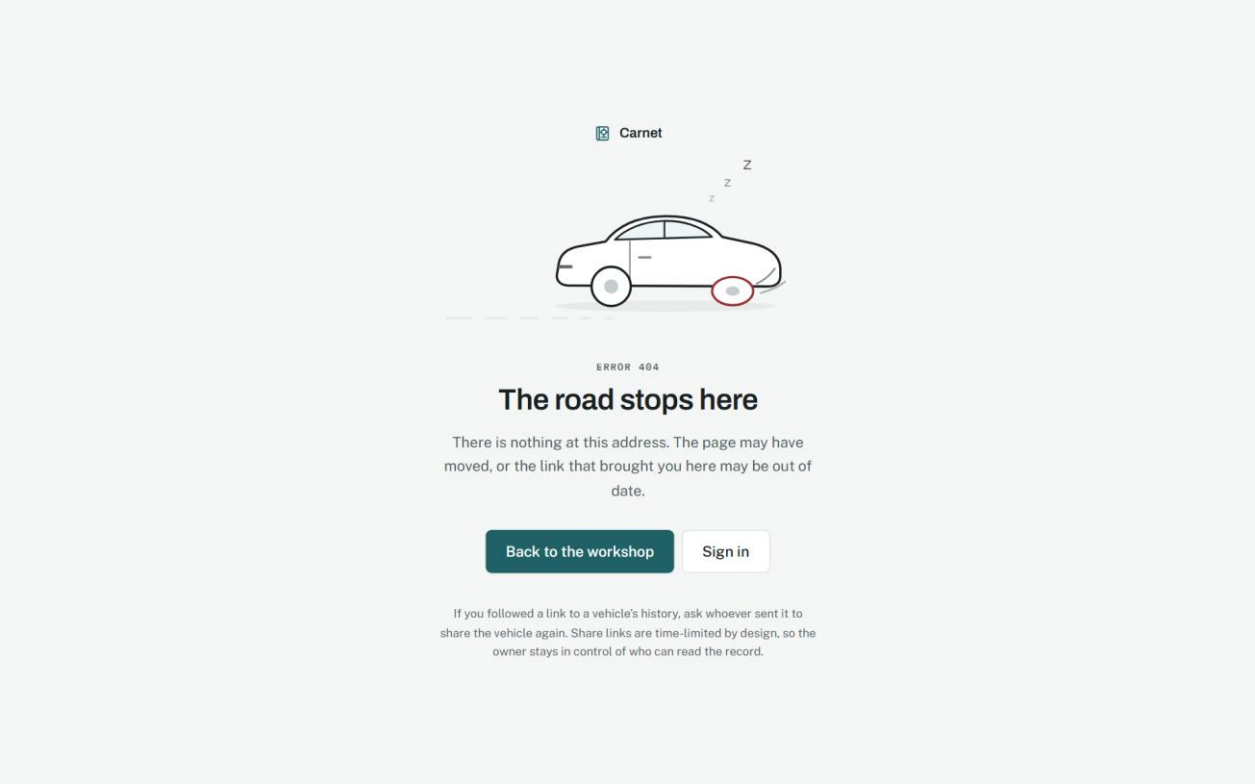
## 2021 Honda Civic LR21 KJD

Nothing recorded yet.

Open the passport

*Every car the customer owns, listed.*

### Public




*Not found, in the product's own voice.*

### Brand



*The mark.*

# 10. What verifies it

Every phase ships behind the same checks: the type checker, the linter, the formatter, the unit tests in packages/core, a production build, and then a built server crawled by three scripts and exercised by the phase's own verification script against a real database. The scripts are in infra/scripts and each removes what it made.

| Script | What it exercises |
|---|---|
| `verify-access` | labels, names, alternatives, heading order, landmarks and skip links on every page |
| `verify-answers` | the capability it is named for, end to end, against a real database |

| | |
|---|---|
| `verify-assistant` | the capability it is named for, end to end, against a real database |
| `verify-authz` | every workshop page as every role, every server action read for its permission, another workshop's records by id, the machine routes with nothing in hand, a read-only workshop, the plan at the point of use |
| `verify-automation` | the capability it is named for, end to end, against a real database |
| `verify-backup` | a backup taken and restored, and the restored database answering |
| `verify-banking` | the capability it is named for, end to end, against a real database |
| `verify-claims` | the capability it is named for, end to end, against a real database |
| `verify-condition` | the capability it is named for, end to end, against a real database |
| `verify-controls` | every control on every page does something and every link goes somewhere |
| `verify-counter` | the capability it is named for, end to end, against a real database |
| `verify-credentials` | the capability it is named for, end to end, against a real database |
| `verify-deliverability` | the capability it is named for, end to end, against a real database |
| `verify-deposits` | money held: taken, applied, returned, kept as credit, and every posting |
| `verify-documents` | the capability it is named for, end to end, against a real database |
| `verify-empty` | every page works with nothing in it, and says what to do next |
| `verify-erp` | the capability it is named for, end to end, against a real database |
| `verify-giftcards` | gift cards sold, spent at the counter, refunded onto the card, lapsed and cancelled |
| `verify-handover` | the capability it is named for, end to end, against a real database |
| `verify-hardening` | the capability it is named for, end to end, against a real database |
| `verify-headers` | the security headers and the content security policy on every surface |
| `verify-inspection` | the capability it is named for, end to end, against a real database |
| `verify-integrity` | the passport chain: append, verify, tamper, detect |
| `verify-inventory` | the capability it is named for, end to end, against a real database |
| `verify-ledger` | postings balance, the trial balance and the statements agree |
| `verify-loyalty` | points earned on payment, spent on a bill, mirrored in the ledger |
| `verify-messaging` | the capability it is named for, end to end, against a real database |
| `verify-money` | the invoice, payment, credit note and refund cycle against a real database |
| `verify-network` | what another workshop may read, and what it may not |
| `verify-offline` | the technician's queue applied through /api/sync, idempotently |
| `verify-passports` | the capability it is named for, end to end, against a real database |
| `verify-payments` | a card payment from the portal: declined with a reason, paid once however often the provider says so, refunded through the provider, reconciled, and the webhook over HTTP |
| `verify-planning` | the capability it is named for, end to end, against a real database |
| `verify-provenance` | the capability it is named for, end to end, against a real database |
| `verify-records` | the capability it is named for, end to end, against a real database |

| | |
|---|---|
| `verify-referrals` | a code redeemed at sign-up and the reward settled on the first paid bill |
| `verify-region` | a workshop signed up in Canada and moved to France, every figure in the right currency |
| `verify-retention` | the capability it is named for, end to end, against a real database |
| `verify-settings` | the capability it is named for, end to end, against a real database |
| `verify-signup` | the capability it is named for, end to end, against a real database |
| `verify-storage` | the capability it is named for, end to end, against a real database |
| `verify-team` | the capability it is named for, end to end, against a real database |
| `verify-warranty` | the capability it is named for, end to end, against a real database |

# Appendix A. Every table, by domain

#### TENANCY, IDENTITY AND THE AUDIT TRAIL

Organization, Location, Bay, User, Membership, Session, LoginAttempt, AuthToken, Invitation, ApiKey, AuditLog, OutboxEvent, SyncedAction

#### THE VEHICLE AND ITS PASSPORT

Vehicle, VehicleEvent, VehicleEventAttachment, VehicleOwnership, OdometerReading, VehicleAccessGrant, PassportShare, PassportShareAccess, Recall, VehicleRecall, VehicleDocument, VehicleClaim, PartFitment

#### CUSTOMERS, BOOKINGS AND THE DIARY

Customer, Tag, Booking, JobType, ServiceItem, ServiceSchedule, FleetAgreement, ConsentChange

#### THE JOB

Job, JobTechnician, JobJobType, JobTag, JobStatusEvent, JobTask, JobLabourLine, JobPartLine, JobFeeLine, JobNote, JobAttachment, TimeEntry, QualityCheck, JobFollowUp

#### INSPECTIONS AND ESTIMATES

InspectionTemplate, InspectionTemplateSection, InspectionTemplateItem, Inspection, InspectionResult, InspectionPhoto, Estimate, EstimateLine, EstimateShare, EstimateApproval

#### INVOICES, PAYMENTS AND REFUNDS

TaxRate, DocumentSequence, Invoice, InvoiceLine, Payment, PaymentAttempt, CreditNote, Refund

#### THE COUNTER, GIFT CARDS AND MONEY HELD

Deposit, DepositMovement, GiftCard, GiftCardEntry, CounterSale, CounterSaleLine, CounterSaleTender, TillSession

#### THE LEDGER, EXPENSES AND BANKING

Account, FiscalPeriod, JournalEntry, JournalLine, Expense, BankAccount, BankStatement, BankStatementLine

#### PARTS AND STOCK

ProductCategory, Product, Warehouse, StockLevel, StockMovement, StockReservation, StockTransfer, StockTransferLine, Warranty, WarrantyClaim

#### SUPPLIERS AND PURCHASING

Supplier, PurchaseRequest, PurchaseOrder, PurchaseOrderLine, GoodsReceipt, GoodsReceiptLine, SupplierBill, SupplierPayment, SupplierPaymentAllocation

#### PEOPLE, SKILLS AND PAYROLL

Employee, PayRate, AttendanceRecord, Timesheet, Absence, Certification, Skill, EmployeeSkill, JobTypeSkill, WorkingPattern, PayrollRun, CommissionScheme, CommissionBand, CommissionAward

#### COMMUNICATION, RETENTION AND AUTOMATION

MessageLog, InboundMessage, SuppressedAddress, DeliveryEvent, ReviewRequest, LoyaltyEntry, Referral, AutomationRule, AutomationRun, AutomationAction, Campaign, CampaignRecipient, AssistantExchange

# Appendix B. Migrations

Hand-written SQL, applied in order by prisma migrate deploy. The name says what it did.

| Migration | Applied as |
|---|---|
| init | 2026-09-09 |
| invariants | 2026-09-09 |
| people_and_multisite | 2026-09-09 |
| people_invariants | 2026-09-09 |
| audit_permits_erasure | 2026-09-09 |
| invitations | 2026-09-10 |
| counter_sales | 2026-09-10 |
| counter_sale_sequence | 2026-09-10 |
| expenses_and_banking | 2026-09-10 |
| automation_and_crm | 2026-09-10 |
| automation_invariants | 2026-09-10 |
| message_queue | 2026-09-10 |
| warranty | 2026-09-11 |
| warranty_invariants | 2026-09-11 |
| skills_and_patterns | 2026-09-11 |
| planning_invariants | 2026-09-11 |
| part_fitment | 2026-09-11 |
| fitment_invariants | 2026-09-11 |
| synced_actions | 2026-09-11 |
| retention | 2026-09-11 |
| auth_tokens | 2026-09-11 |
| deliverability | 2026-09-12 |
| vehicle_claims | 2026-09-12 |
| refunds | 2026-09-12 |
| payables_payroll_fleet | 2026-09-12 |
| api_keys | 2026-09-12 |
| opening_hours_and_plan | 2026-09-13 |
| vehicle_documents | 2026-09-13 |
| api_rate_limit | 2026-09-13 |
| assistant | 2026-09-13 |
| loyalty_payments | 2026-09-13 |

| | |
|---|---|
| gift_cards | 2026-09-14 |
| gift_cards_deepened | 2026-09-14 |
| organisation_country | 2026-09-14 |
| deposits | 2026-09-14 |
| subscription | 2026-09-14 |
| card_payments | 2026-09-14 |

# Appendix C. Rebuilding this document

```
npm run build --workspace=@carnet/web && npm run start --workspace=@carnet/web   # a built server on
:3000
npm run docs:shots         # photographs of the running app  -> docs/assets/design/*.png
npm run docs:diagrams      # ER and flow diagrams from the schema -> docs/assets/architecture/*.png
npm run docs:architecture  # this document                      -> docs/Carnet-System-Architecture.docx /
.pdf
npm run docs:all           # the deck, the business case, the design book and this document
```

A model added to the schema must be placed in a domain in infra/scripts/architecture-diagrams.ts, or the diagram build fails and says which. A screen added to the app should be added to design-shots.ts and captioned here. The prose in build_architecture_doc.py is revised with each phase.